6 Gestione della qualità del software

OBIETTIVI FORMATIVI

- Apprezzare la necessità di misurazioni corrette per determinare la qualità del software
- Valutare criticamente varie tassonomie di attributi di qualità
- Essere in grado di confrontare diversi punti di vista della qualità del software
- Essere a conoscenza degli standard internazionali relativi alla qualità del software
- Conoscere il Capability Maturity Model (modello di maturità delle capacità)
- Capire come un'organizzazione può impostare il proprio programma di misurazione

La qualità del software è un argomento importante. Con la crescente penetrazione dell'automazione nella vita di tutti i giorni, sempre più persone entrano in contatto con i sistemi software e la qualità di tali sistemi è una delle principali preoccupazioni. La qualità non può essere aggiunta come un ripensamento. Deve essere integrata fin dall'inizio. Questo capitolo discute le numerose dimensioni della qualità sia del **prodotto** software che del **processo** software.

Nel loro libro di riferimento, *Alla ricerca dell'eccellenza,* Peters e Waterman identificano una serie di fattori chiave che distinguono le aziende di grande successo del mondo da quelle di minor successo. Uno di questi fattori chiave è l'impegno per la qualità delle aziende di grande successo. A quanto pare, la qualità paga.

La redditività a lungo termine non è l'unico motivo per cui l'attenzione alla qualità è importante nello sviluppo del software. A causa dell'assoluta complessità dei prodotti software e dei cambiamenti spesso frequenti che devono essere incorporati durante lo sviluppo del software, è necessaria una continua attenzione e valutazione della qualità del prodotto in fase di sviluppo se vogliamo realizzare prodotti soddisfacenti. Questa esigenza è aggravata dalla crescente penetrazione della tecnologia software nella vita di tutti i giorni. I prodotti di bassa qualità lasceranno i clienti insoddisfatti, faranno trascurare agli utenti i sistemi che dovrebbero supportare il loro lavoro e potrebbero persino costare vite umane.

Un esempio spaventoso di ciò che potrebbe accadere se il software contiene dei bug, è diventato noto come 'Malfunction 54'. Il Therac-25, una macchina radioattiva computerizzata, è stato accusato di incidenti che hanno causato la morte di due persone e gravi ferite ad altre. Il mistero mortale è stato infine ricondotto a un bug del software, chiamato "Malfunction 54" dopo il messaggio visualizzato sulla console; vedere anche **la Sezione 1.4.2**. L'impegno per la qualità nello sviluppo del software non solo ripaga, ma è una vera e propria necessità.

Questo impegno richiede attenti processi di sviluppo. Questa attenzione al **processo di sviluppo** si basa sul presupposto che la qualità di un **prodotto** è in gran parte basata sulla qualità del processo che porta a quel prodotto e che questo processo può effettivamente essere definito, gestito, misurato e migliorato.

Oltre alla dicotomia prodotto-processo, si può distinguere anche una dicotomia tra **conformità** e **miglioramento**. Se imponiamo determinati requisiti di qualità al prodotto o processo, possiamo escogitare tecniche e procedure per garantire o testare che il prodotto o processo sia effettivamente *conforme a* questi obiettivi. In alternativa, alcuni schemi possono essere volti a *migliorare* la qualità del prodotto o del processo.

La tabella 6.1 fornisce esempi di questi quattro approcci alla qualità. La maggior parte dell'ingegneria del software si occupa del miglioramento della qualità dei prodotti che sviluppiamo e l'etichetta "migliori pratiche" in questa figura si riferisce a tutte le chicche menzionate altrove in questo libro. Gli altri tre approcci sono discussi in questo capitolo.

Prima di intraprendere una discussione sui diversi approcci alla qualità, elaboreremo innanzitutto la nozione di qualità del software stessa e come misurarla. Quando si parla dell'altezza di una persona, la frase "Jasper è altro 195 cm" trasmette più informazioni di "Jasper è alto". Allo stesso modo, vorremmo esprimere in numeri tutti i tipi di attributi di qualità. Preferiremmo una dichiarazione del modulo "La disponibilità del sistema è del 99%" a un semplice "La disponibilità del sistema è elevata". Alcuni degli avvertimenti delle questioni relative alla misurazione sono discussi nella **Sezione 6.1.** Nella Sezione 6.2, discutiamo varie tassonomie di attributi di qualità, incluso ISO 9126. Questa non è affatto l'ultima parola sulla qualità del software, ma è un buon punto di riferimento da cui partire. Questa discussione ci permette anche di illustrare ulteriormente alcuni dei problemi con la misurazione della qualità in termini quantitativi.

Tavola 6-1 Approcci alla qualità

	Conformità	Miglioramento
Prodotto	ISO 9126	'migliori pratiche' (best practices)
Processi	ISO 9001 SQA	CMM SPICE Bootstrap

La "qualità del software" è un concetto piuttosto sfuggente. Persone diverse avranno prospettive diverse sulla qualità di un sistema software. Un tester di sistema può vedere la qualità come "conformità ai requisiti", mentre un utente può vederla come "idoneità all'uso". Entrambi i punti di vista sono validi, ma non devono necessariamente coincidere. In effetti, probabilmente non lo faranno. Parte della confusione su cosa comporti la qualità di un sistema e come dovrebbe essere valutata è causata dal confondere queste diverse

prospettive. Piuttosto che differenziare tra le varie prospettive sulla qualità, il Total Quality Management (TQM) sostiene una visione eclettica: la qualità è la ricerca dell'eccellenza in ogni cosa. La sezione 6.3 approfondisce le diverse prospettive sulla qualità.

L'Organizzazione internazionale per la standardizzazione (ISO) ha stabilito diversi standard relativi alla gestione della qualità. Quello più applicabile al nostro campo, lo sviluppo e la manutenzione del software, è **ISO 9001**. Questo standard è discusso nella **Sezione 6.4**.

La ISO 9001 può essere integrata da procedure mirate specificamente alla garanzia e al controllo della qualità per lo sviluppo del software. Lo standard IEEE per i piani di garanzia della qualità ha lo scopo di fornire tali procedure. È discusso nella **Sezione 6.5**. Le procedure di garanzia della qualità del software forniscono i mezzi per rivedere e verificare il processo di sviluppo del software e dei suoi prodotti. La garanzia della qualità di per sé non garantisce prodotti di qualità. La garanzia della qualità si limita a fare in modo che il lavoro sia svolto nel modo in cui dovrebbe essere svolto.

Il Capability Maturity Model (CMM) [1] è il tentativo più noto di indicazioni su come migliorare il processo di sviluppo. Utilizza una scala di cinque punti per valutare le organizzazioni e indica le aree chiave di interesse per passare a un livello di maturità più elevato. SPICE e Bootstrap sono approcci simili al miglioramento dei processi. CMM è discusso nella Sezione 6.6 .

Le azioni per la qualità all'interno delle organizzazioni di sviluppo software mirano a trovare opportunità per migliorare il processo di sviluppo. Questi miglioramenti richiedono una comprensione del processo di sviluppo, che può essere ottenuto solo attraverso un'accurata raccolta e interpretazione dei dati che riguardano gli aspetti qualitativi del processo e dei suoi prodotti. Alcuni suggerimenti su come avviare un tale programma di miglioramento della qualità sono forniti nella Sezione 6.7 .

6.1 SU MISURE E NUMERI – METRICHE DEL SOFTWARE

- *Quando puoi misurare ciò di cui stai parlando ed esprimerlo in numeri, ne sai qualcosa; ma quando non puoi misurarlo, quando non puoi esprimerlo in numeri, la tua conoscenza è di un genere scarso e insoddisfacente; può essere l'inizio della conoscenza, ma nei tuoi pensieri sei appena avanzato allo stadio della scienza.*
  Lord Kelvin (1900)
- *È segno di una mente istruita rimanere soddisfatto del grado di precisione che la natura di un soggetto ammette e non cercare l'esattezza quando è possibile solo un'approssimazione della verità.*
  Aristotele (330 a.C.)

Supponiamo di voler esprimere un attributo di qualità, diciamo la complessità del testo di un programma, in un unico valore numerico. Valori più grandi indicano programmi più complessi. Se è possibile trovare una tale mappatura C dai programmi ai numeri, possiamo quindi confrontare i valori di C (P_1) e C (P_2) per decidere se il programma P_1 è più complesso del

programma P_2. Poiché i programmi più complessi sono più difficili da comprendere e mantenere, questo tipo di informazioni è molto utile, ad esempio per pianificare le attività di manutenzione.

Quale dovrebbe essere allora questa mappatura? Considera i testi del programma in **Figura 6.1** .

La maggior parte delle persone concorderà sul fatto che il testo (a) sembri meno complesso del testo (b). È causato da:

- la sua lunghezza,
- il numero di istruzioni goto,
- il numero di istruzioni if,
- una combinazione di questi attributi,
- qualcos'altro?

Supponiamo di decidere che ciò che conta è il numero delle istruzioni if. Il risultato della mappatura è 0 per il testo (a) e 3 per il testo (b), e questo concorda con la nostra intuizione. Tuttavia, se prendiamo la somma del numero di istruzioni if, goto e loop, anche il risultato concorda con la nostra intuizione. Quale di queste mappature è quella ricercata? Uno dei due è "valido" per cominciare? Cosa significa "valido" in questo contesto?

(a)

```
1   procedure bubble
2       (var a: array [1..n] of integer; n: integer);
3   var i, j, temp: integer;
4   begin
5       for i:= 2 to n do
6           j:= i;
7           while j > 1 and a[j] < a[j-1] do
8               temp:= a[j];
9               a[j]:= a[j-1];
10              a[j-1]:= temp;
11              j:= j-1;
12          enddo
13      enddo
14  end;
```

(b)

```
1   procedure bubble
2       (var a: array [1..n] of integer; n: integer);
3   var i, j, temp: integer;
4   begin
5       for i:= 2 to n do
6           if a[i] ≥ a[i-1] then goto next endif;
7           j:= i;
8       loop: if j ≤ 1 then goto next endif;
9           if a[j] ≥ a[j-1] then goto next endif;
10          temp:= a[j];
11          a[j]:= a[j-1];
12          a[j-1]:= temp;
13          j:= j-1;
14          goto loop;
15      next: skip;
16      enddo
17  end;
```

Figura 6.1Due versioni di una routine di ordinamento (a) strutturata e (b) non strutturata

Un certo numero di aspetti rilevanti della misurazione, come attributi, unità e tipi di scala, possono essere introdotti e messi in relazione l'uno con l'altro utilizzando la struttura di misurazione illustrata nella **Figura 6.2** . Questo framework ci consente anche di indicare

come le metriche possono essere utilizzate per descrivere e prevedere le proprietà di prodotti e processi e come convalidare queste previsioni.

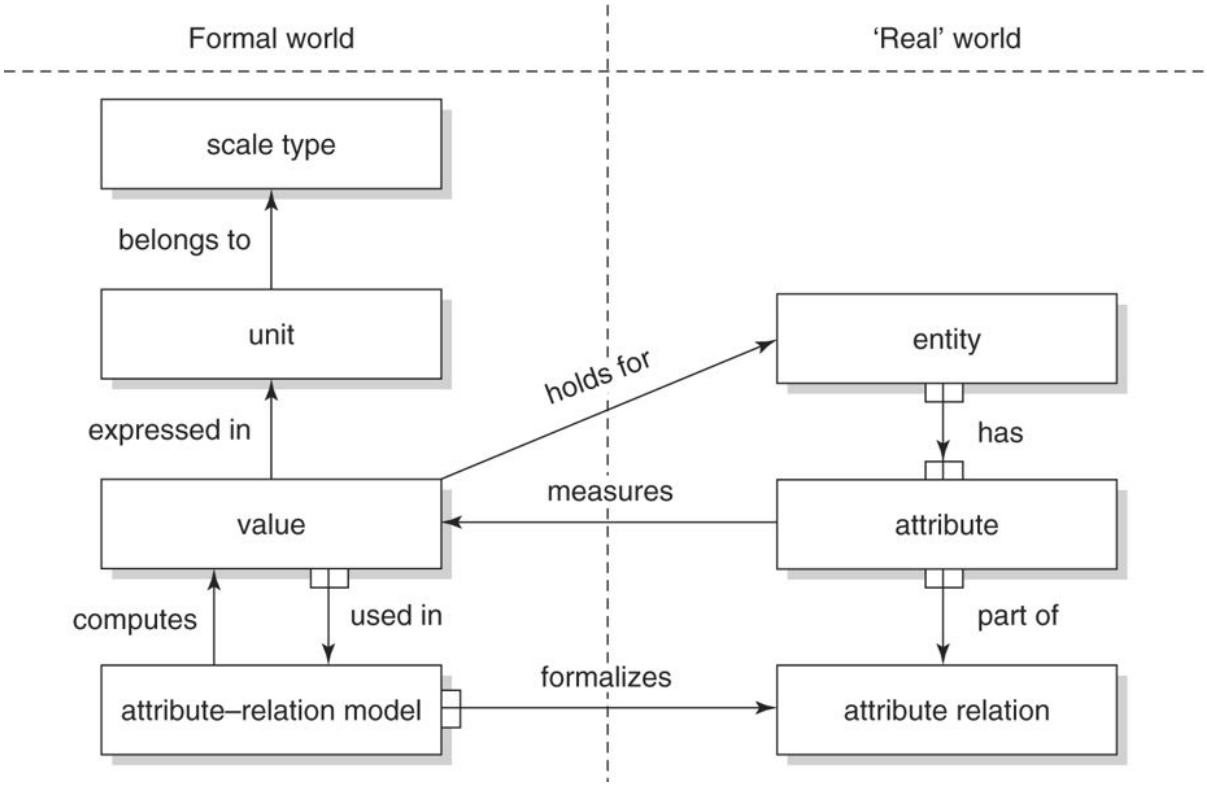


Figura 6.2 Un framework di misurazione (Fonte: B. Kitchenham, S. Pfleeger e N. Fenton, Towards a Framework for Software Measurement Validation, IEEE Transactions on Software Engineering 21:12, © 1995 IEEE. Riprodotto con autorizzazione.)

Il modello in Figura 6.2 ha sette costituenti:

- **Entità** Un'entità è un oggetto nel mondo 'reale' di cui vogliamo conoscere o prevedere determinate proprietà. Le entità non devono necessariamente denotare oggetti materiali; progetti e software sono entità.
- **Attributo:** le entità hanno determinate proprietà che chiamiamo attributi. Entità diverse possono avere lo stesso attributo: sia le persone che le auto hanno un peso. E, naturalmente, una singola entità può avere più di un attributo. Le forcelle che adornano la freccia etichettata 'ha' nella **Figura 6.2** indicano che questa relazione è molti a molti.
- **Relazioni tra attribuiti:** diversi attributi di una o più entità possono essere in relazione. Ad esempio, gli attributi 'lunghezza' e 'peso' di un'entità 'serpente' sono correlati. Allo stesso modo, il numero di mesi uomo spesi per un progetto è correlato al costo di quel progetto. Inoltre, un attributo di un'entità può essere correlato a un attributo di un'altra entità. Ad esempio, L'esperienza di un programmatore può essere correlata al costo di un progetto di sviluppo su cui sta lavorando.
- **Valore:** Si vogliono caratterizzare formalmente i costituenti del modello che risiedono nel mondo 'reale' *misurando* gli attributi, ovvero assegnando loro dei valori.

- **Unità:** Ovviamente il valore di un attributo è espresso in una determinata unità, come metri, secondi o righe di codice.
- **Tipi di scala:** L'unità a sua volta appartiene a un determinato tipo di scala. Alcuni tipi di scala comuni sono i seguenti.

Tipi di scale

- Gli attributi **nominali** sono solo classificati: il colore dei miei capelli è grigio, bianco o nero. La relazione tra i valori consente una semplice classificazione degli oggetti della misurazione, ma non ci indica nulla sulla relazione fra di essi
    - Esempio: la classificazione degli errori di programmazione in lessicali, sintattici e semantici ci consente di suddividerli ma non di sapere se un errore sia più o meno grave di un altro;
    - in termini più analitici questo tipo di scala coincide con l'introduzione nell'insieme di partenza di classi di equivalenza
    - Operazioni applicabili: Moda
- **Ordinale** Esiste un ordinamento (lineare) nei possibili valori di un attributo: un tipo di materiale è più duro di un altro, un programma è più complesso di un altro.
    - si introduce una relazione d'ordine fra gli oggetti, per cui si è in grado di determinare le posizioni relative degli oggetti (cioè dire se uno venga prima di un altro);
    - non si è però in grado di quantificare la distanza fra gli oggetti
    - Operazioni applicabili: Mediana
    - Esempio: in una scala di valutazione (sufficiente, buono, ottimo) possiamo dire che buono è peggiore di ottimo ma non di sufficiente
- **Intervallo** Esiste un ordinamento (lineare) ma la 'distanza' tra i valori successivi di un attributo è la stessa, come in un calendario o in una temperatura misurata in gradi Fahrenheit.
    - Si è in grado di misurare la distanza fra gli oggetti del dominio, e non solo posizionarli tra di loro come con le scale ordinali
    - Operazioni applicabili: Media Aritmetica
    - Esempio: la percentuale di righe di commenti in un software può essere "1%", "7%" e "10%" in sostituzione di "sufficiente", "buono" e "ottimo", rispettivamente, consente non solo di dire che "10%" è maggiore di "7%", ma anche di quanto
- **Rapporto (ratio)** Esiste un ordinamento (lineare) con uguale 'distanza' tra valori successivi, con l'ulteriore requisito che esista un valore 0, come nell'età di un sistema software o nella temperatura misurata in gradi Kelvin.
    - introduce l'elemento zero che rappresenta l'assenza totale dell'attributo che si sta misurando nell'entità sottoposta a misurazione. L'introduzione dello zero, inoltre, conferisce al valore di un attributo il senso di positività, negatività o nullità.

    - Esempio: la misura di una temperatura, quantificata con un numero maggiore, minore, o uguale a zero.
- **Assoluto** In questo caso contiamo semplicemente il numero di occorrenze.
    - esiste una strategia di conteggio per la quale possiamo assegnare ad ogni oggetto un numero univocamente determinato.
    - La scala assoluta è usata per quegli attributi di un oggetto che richiedono un semplice conteggio di elementi.
    - – Esempio: si consideri il numero di errori rilevati in un software e si voglia quantificare il suo attributo “numero di errori”. La misura in questo caso consiste nel semplice conteggio degli errori; il totale conteggiato rappresenta la erroneità del software.

Nota che a volte possiamo misurare un attributo in unità diverse, dove queste unità si trovano su scale diverse. Ad esempio, possiamo misurare la temperatura su una scala ordinale: o si congela o no. Possiamo anche misurarlo su una scala di intervallo, in gradi Fahrenheit o Celsius, e possiamo misurarlo su una scala di rapporti, in gradi Kelvin.

- **Modello attributo-relazione** Se esiste una relazione tra diversi attributi di, possibilmente differenti, entità nel mondo 'reale', possiamo esprimere tale relazione in un modello formale, il modello attributo-relazione. Questo modello calcola (predice) il valore di un attributo a cui siamo interessati dai valori di uno o più altri attributi del modello. Il fork sulla freccia etichettata 'formalizza' nella **Figura 6.2** indica che possiamo avere più di un modello per la stessa relazione di attributo.

La misurazione è una mappatura dal mondo empirico, "reale" al mondo formale e relazionale. Una **misura** è il numero o il simbolo assegnato a un attributo di un'entità da questa mappatura. Il valore assegnato ha ovviamente una certa unità, es. righe di codice. L'unità a sua volta appartiene a una certa scala, come la scala del rapporto per le righe di codice o la scala ordinale per la gravità di un errore.

In matematica, il termine **metrica** ha un significato molto specifico: descrive quanto sono distanti due punti. Nel nostro campo, il termine è spesso usato in modo un po' sciatto. A volte denota una misura, a volte l'unità di misura. Usiamo il termine per indicare la combinazione di:

- un attributo di un'entità,
- la funzione che assegna un valore a quell'attributo,
- l'unità in cui è espresso questo valore, e
- il suo tipo di scala.

Per ogni tipo di scala sono consentite determinate operazioni, altre no. In particolare, non possiamo calcolare la media per una scala ordinale, ma solo la sua mediana (valore medio). Supponiamo di classificare un sistema come "molto complesso", "complesso", "medio", "semplice" o "molto semplice". L'assegnazione di numeri a questi valori è piuttosto arbitraria.

L'unico prerequisito è che a un sistema classificato come 'molto complesso' venga assegnato un valore maggiore rispetto a un sistema classificato come 'complesso'. Se chiamiamo questa mappatura *W,* l'unico requisito è: *W* (molto complesso) > *W* (complesso) > ⋯ > *W* (molto semplice). La Tabella 6.2 fornisce un esempio di due assegnazioni valide di valori a questo attributo.

Tabella 6-2 Mappature di esempio per una scala ordinale

Molto complesso	Complesso	Media	Semplice	Molto semplice
5	4	3	2	1
100	10	5	2	1

Supponiamo di avere un sistema con tre componenti, che sono caratterizzati rispettivamente come 'molto complesso', 'medio' e 'semplice'. Assegnando i valori della prima riga, la media sarebbe 3, quindi l'intero sistema sarebbe classificato come di complessità media. Usando i valori della seconda riga, la media sarebbe 35, qualcosa tra "complesso" e "molto complesso". Il problema è causato dal fatto che, con una scala ordinale, non sappiamo se i valori successivi siano *equidistanti.* Quando calcoliamo una media, assumiamo tacitamente che lo siano.

6.1.1 Metriche del software

Metrica: *misura quantitativa del grado di possesso di uno specifico attributo da parte di un sistema, un componente, o un processo* [IEEE Std 610.12-1990: IEEE Standard Glossary of Software Engineering Terminology]

– Es: numero di linee di codice di un modulo, numero di casi d'uso di un'applicazione,...

- Molto spesso, **metrica** è usato come sinonimo di misura o misurazione
- la definizione di metrica include procedure e **modalità** di misurazione.

Misurazione diretta e indiretta

Alcune volte non possiamo misurare *direttamente* il valore di un attributo. Ad esempio, la velocità di un'auto può essere determinata dai valori di altri due attributi: una distanza e il tempo impiegato dall'auto per percorrere quella distanza. La velocità viene quindi misurata *indirettamente,* prendendo il quoziente di due misure dirette. In questo caso, il modello attributo-relazione formalizza la relazione tra la distanza percorsa, il tempo e la velocità.

6.1.2 Attributi interni ed esterni

Possiamo distinguere tra attributi *interni* ed *esterni.* Gli attributi interni di un'entità possono essere misurati esclusivamente in termini di quell'entità stessa. Modularità, dimensioni, difetti riscontrati e costi sono esempi tipici di attributi interni. Gli attributi esterni di un'entità

sono quelli che possono essere misurati solo rispetto al modo in cui quell'entità si relaziona al suo ambiente**. Manutenibilità e usabilità sono esempi di attributi esterni**. La maggior parte dei fattori di qualità di cui discutiamo in questo capitolo sono attributi esterni. Gli attributi esterni possono essere misurati solo *indirettamente,* poiché implicano la misurazione di altri attributi.

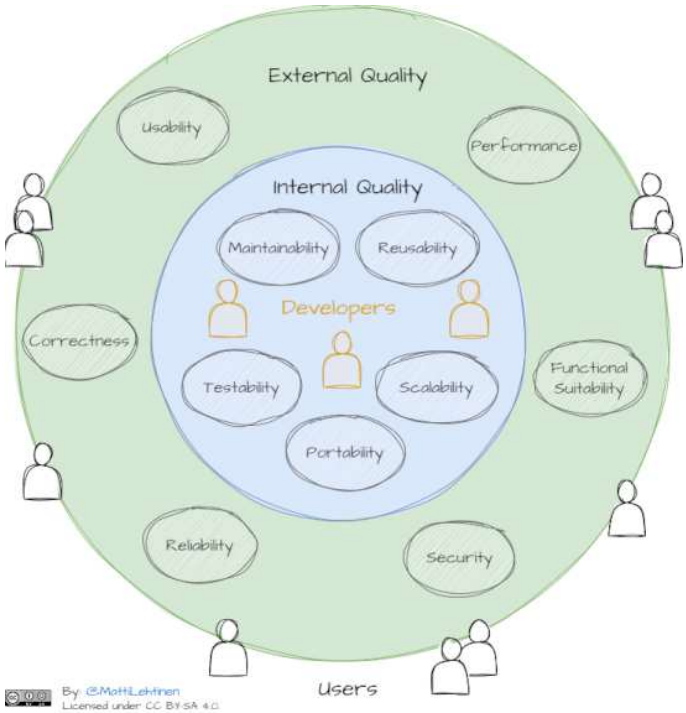
https://mattilehtinen.com/articles/internal-vs-external-software-quality-and-how-to-improve-them/

Immaginate una splendida villa con una facciata imponente che sembra un sogno che si avvera. A un esame più attento, tuttavia, le sue fondamenta stanno cedendo e la struttura sta quasi cadendo a pezzi. Si potrebbe dire che ha un esterno di alta qualità, ma una struttura interna di bassa qualità. Allo stesso modo, esistono due tipi di qualità del software: esterna e interna.

Il software può essere molto facile da usare e potrebbe funzionare velocemente (qualità esterna), ma potrebbe essere difficile aggiungervi nuove funzionalità a causa del codice spaghetti disordinato (qualità interna).

Può anche accadere il contrario: la funzionalità del software può essere facilmente modificata (qualità interna), ma le sue prestazioni possono essere pessime e il software potrebbe bloccarsi frequentemente (qualità esterna).

Il diagramma seguente visualizza la qualità interna ed esterna, includendo esempi di entrambe:



La qualità **esterna** è il modo in cui gli **utenti** percepiscono il software o il prodotto. Ad esempio, potrebbero desiderare che sia facile da usare, funzioni come previsto e che le cose vengano eseguite rapidamente. Di solito, quando si sente parlare di qualità di un'applicazione, ci si riferisce alla qualità esterna.

La qualità **interna** è ciò di cui gli **sviluppatori** sono consapevoli e a cui tengono. Si tratta di aspetti che non si possono vedere o sperimentare direttamente durante l'utilizzo del software, ma che incidono sullo sviluppo e sui costi del software. Ad esempio, la manutenibilità e la leggibilità del codice sono aspetti della qualità interna.

Qualità del software esterno

La qualità esterna è la qualità percepita del sistema che gli utenti sperimentano quando utilizzano il software. È qualcosa a cui gli utenti prestano attenzione e a cui sono interessati.

Di solito, quando gli utenti parlano della qualità di un software, si riferiscono alla qualità esterna. Ad esempio, possono dire cose come:

"Adoro quanto sia facile usare questo sito web!"

"Questa app si blocca continuamente!"

"Ci vuole un'eternità perché questa pagina si carichi..."

Questi sono tutti aspetti della qualità esterna, e molti altri fattori possono influenzare la qualità percepita.

Qualità interna del software

La qualità interna del software è qualcosa che i tuoi utenti non possono sperimentare o vedere direttamente, ma è qualcosa a cui dovresti essere interessato. Si riferisce alla qualità della base di codice, del design e dell'architettura del software e quindi può influire direttamente sull'efficienza dello sviluppo del software.

Ad esempio, un codice "spaghetti" difficile da leggere è segno di scarsa qualità interna. Il codice dovrebbe essere pulito, autodescrittivo e scritto principalmente per gli esseri umani.

6.1.3 Validità di una misura

Le relazioni empiriche tra oggetti nel mondo reale dovrebbero essere preservate nel sistema di relazioni numeriche che utilizziamo. Se osserviamo che l'auto *A* guida più velocemente dell'auto *B,* allora vorremmo piuttosto che la nostra funzione *S,* che mappa le velocità osservate su un certo numero, fosse tale che $S(A) > S(B)$. Questa è chiamata **condizione di rappresentazione.** Se una misura soddisfa la condizione di rappresentazione, si dice **misura valida.**

La condizione di rappresentazione può talvolta essere verificata da un'attenta valutazione del modello attributo-relazione. Ad esempio, in precedenza abbiamo proposto di misurare la

complessità del testo di un programma contando il numero di istruzioni if. Affinché questa misura (indiretta) sia valida dobbiamo accertare che:

- due programmi qualsiasi con lo stesso numero di istruzioni if sono ugualmente complessi, e
- se il programma *A* ha più istruzioni if del programma *B,* allora *A* è più complesso di *B.*

Poiché nessuna di queste affermazioni è vera nel mondo reale, questa misura di complessità non è valida.

La validità di misure indirette più complesse viene generalmente accertata attraverso mezzi statistici. La maggior parte dei modelli di stima dei costi discussi nel **Capitolo 7**, ad esempio, vengono convalidati in questo modo.

Infine, il tipo di scala delle misure indirette merita una certa attenzione. Se diverse misure vengono combinate in una nuova misura, il tipo di scala della misura combinata è il "più debole" dei tipi di scala dei suoi componenti. Molte formule di stima dei costi contengono fattori il cui tipo di scala è ordinale, come l'instabilità dei requisiti o l'esperienza del team di progettazione. A rigor di termini, i diversi valori che risultano dall'applicazione di tale formula di stima dei costi dovrebbero quindi essere interpretati come indicanti che alcuni progetti richiedono uno sforzo maggiore di altri. L'intenzione però è di interpretarli su una scala di rapporti, cioè lo sforzo effettivo in mesi uomo. Dal punto di vista della teoria della misurazione, questo non è consentito.

6.1.4 Metriche di sforzo

Per sforzo (**effort**) si intende la misura del quantitativo di lavoro umano necessario per svolgere un lavoro relativo ad un processo di sviluppo software

L'unità di misura più comunemente usata è il mese/uomo (man-month) e tutti i suoi multipli e sottomultipli: Un mese/uomo si definisce, teoricamente, come il quantitativo di lavoro che un singolo dipendente può svolgere in un mese lavorativo. L'uso dei mesi uomo per quantificare lo sforzo presenta alcune criticità.

– Ogni persona ha una sua diversa produttività, che può variare nel tempo (con l'età e la conoscenza, ad esempio).

– La produttività di una persona dipende dalla complessità del problema da risolvere

– Aumentando la dimensione del gruppo di lavoro aumenta proporzionalmente la produttività? Se una persona può svolgere 1 man-month di lavoro in un mese, quanto lavoro potranno svolgere 10 persone nello stesso mese?

Tutti questi problemi furono espressi compiutamente per la prima volta da Brooks nel famoso libro The Mythical Man-Month: Essays on Software Engineering, nel 1975. In conclusione: si tratta di una metrica di facile misura (ma solo a posteriori) ma di difficilissima stima.

6.2 UNA TASSONOMIA DEGLI ATTRIBUTI DI QUALITÀ

Alcuni dei primi elaborati studi ((McCall *et al.*, **1977**) e (Boehm *et al.*, **1978**)) sulla nozione di "qualità del software" sono apparsi alla fine degli anni '70. In questi studi vengono studiati numerosi aspetti dei sistemi software che in qualche modo si riferiscono alla nozione di qualità del software. Negli anni seguenti, un gran numero di persone ha cercato di affrontare questo problema. Sono state pubblicate molte tassonomie di fattori di qualità. Tuttavia, i problemi fondamentali non sono stati risolti in modo soddisfacente. I vari fattori che si riferiscono alla qualità del software sono difficili da definire. È ancora più difficile misurarli quantitativamente. D'altra parte, la qualità reale può spesso essere identificata in modo sorprendentemente facile.

Nel *Glossario IEEE della terminologia dell'ingegneria del software,* la **qualità** è definita come "il grado in cui un sistema, componente o processo soddisfa le esigenze o le aspettative del cliente o dell'utente". Applicata al software, quindi, la qualità dovrebbe essere misurata principalmente rispetto al grado di soddisfazione dei requisiti dell'utente: correttezza, affidabilità, usabilità e così via. Il software dura a lungo e viene adattato di volta in volta per adattarsi alle mutate circostanze. È importante per l'utente che ciò sia possibile a costi ragionevoli. Il cliente è quindi interessato anche ai fattori di qualità che riguardano la struttura del sistema, come manutenibilità, testabilità e portabilità, piuttosto che al suo utilizzo.

Inizieremo la nostra discussione sugli attributi di qualità con la tassonomia di McCall, che distingue tra due livelli di attributi di qualità.

Fattori di Qualità: Gli attributi di qualità di livello superiore che possono essere accessibili direttamente sono chiamati fattori di qualità. Questi attributi sono esterni. Gli attributi a questo livello ricevono maggiore importanza da parte degli utenti e dei manager.

Criteri di Qualità: Gli attributi di qualità di livello inferiore o di secondo livello che possono essere accessibili sia soggettivamente sia oggettivamente sono chiamati Criteri di Qualità. Questi attributi sono interni. Ogni fattore di qualità ha molti attributi di qualità o criteri di qualità di secondo livello. I programmatori tendono a dare importanza ai criteri di qualità.

Esempio: Il fattore di qualità dell'usabilità è suddiviso in operabilità, addestramento, comunicazione, volume di input/output e tasso di input/output.

Questo modello classifica tutti i requisiti software in 11 fattori di qualità software. Gli 11 fattori sono organizzati in tre fattori di qualità del prodotto: **Funzionamento del prodotto**, **revisione del prodotto** e **transizione del prodotto.**

I fattori di qualità sono esterni e indiretti. Ad esempio, non possiamo misurare direttamente l'affidabilità di un sistema software. Possiamo tuttavia misurare direttamente il numero di difetti riscontrati finora. Questa misura diretta può essere utilizzata per ottenere informazioni sull'affidabilità del sistema. Ciò implica una teoria, che può essere accertata su basi valide, di come il numero di difetti sia correlato all'affidabilità. Per la maggior parte degli altri aspetti

della qualità, tuttavia, la relazione tra gli attributi che possono essere misurati direttamente e gli attributi esterni a cui siamo interessati è a dir poco meno ovvia.

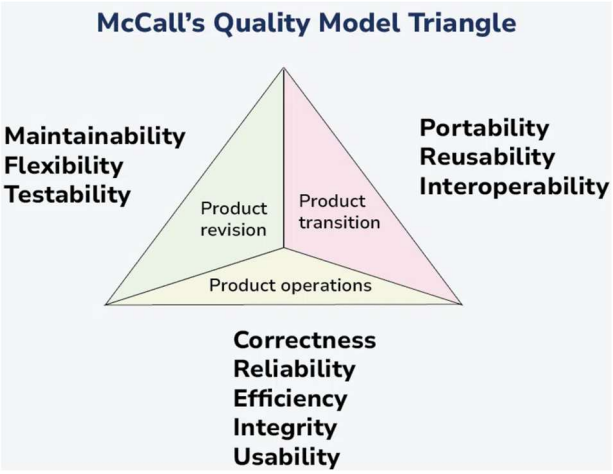
McCall considera tutti I fattori di qualità come esterni perché si focalizzano sulla qualità come percepita da utenti e manager. Alcuni di questi (ad esempio manutenibilità) potrebbero essere anche considerati interni (specifici del prodotto software).

Tabella 6-3 Fattori di qualità

PRODUCT OPERATIONS - Funzionamento del prodotto	
Correttezza	La misura in cui un programma soddisfa le sue specifiche e soddisfa gli obiettivi della missione dell'utente.
Affidabilità	La misura in cui ci si può aspettare che un programma svolga la sua funzione prevista con la precisione richiesta.
Efficienza	La quantità di risorse di calcolo e di codice richiesta da un programma per eseguire una funzione.
Integrità	La misura in cui è possibile controllare l'accesso al software o ai dati da parte di persone non autorizzate.
Usabilità	Lo sforzo richiesto per apprendere, utilizzare, preparare l'input e interpretare l'output di un programma.
PRODUCT REVISION - revisione del prodotto	
Manutenibilità	Lo sforzo necessario per individuare e correggere un errore in un programma operativo.
Testabilità	Lo sforzo richiesto per testare un programma per assicurarsi che svolga la sua funzione prevista.
Flessibilità	Lo sforzo necessario per modificare un programma operativo.
PRODUCT TRANSITION - transizione del prodotto	
Portabilità	Lo sforzo richiesto per trasferire un programma da un ambiente hardware o software a un altro.
Riutilizzabilità	La misura in cui un programma (o parti di esso) può essere riutilizzato in altre applicazioni.
Interoperabilità	Lo sforzo richiesto per accoppiare un sistema con un altro.

(*Fonte:* JA McCall, PK Richards e GF Walters, Factors in Software Quality, RADC-TR-77-369, © 1977, Dipartimento del Commercio degli Stati Uniti.)

Tabella 6.3 elenca **i fattori di qualità** e le loro definizioni, così come sono usati in (McCall *et al.*, 1977) - https://www.geeksforgeeks.org/software-engineering/mccalls-quality-model/



La definizione di affidabilità del software nella Tabella 6.3 è piuttosto ristretta. Una definizione più completa è contenuta nel *Glossario IEEE della terminologia dell'ingegneria del software:* "La capacità di un sistema o componente di eseguire le funzioni richieste in condizioni stabilite per un periodo di tempo specificato". È spesso espresso come una probabilità.

I fattori di qualità possono essere classificati in tre classi. La prima classe contiene quei fattori che riguardano l'uso del software dopo che è diventato **operativo**. La seconda classe riguarda la **manutenibilità** del sistema. La terza classe contiene fattori che riflettono la facilità con cui è possibile effettuare una **transizione** verso un nuovo ambiente. Queste tre categorie sono illustrate nella Tabella 6.4 .

Tabella 6-4 Categorie di fattori di qualità del software

Funzionamento del prodotto:		
	Correttezza	Fa quello che voglio?
	Affidabilità	Lo fa sempre accuratamente?
	Efficienza	Funzionerà sul mio hardware nel miglior modo possibile?
	Integrità	È sicuro?
	Usabilità	Posso eseguirlo?
Revisione del prodotto:		
	Manutenibilità	Posso aggiustarlo?
	Testabilità	Posso testarlo?
	Flessibilità	Posso cambiarlo?
Transizione del prodotto:		
	Portabilità	Potrò usarlo su un'altra macchina?
	Riutilizzabilità	Potrò riutilizzare parte del software?
	Interoperabilità	Potrò interfacciarlo con un altro sistema?

(*Fonte:* JA McCall, PK Richards e GF Walters, Factors in Software Quality, RADC-TR-77-369, © 1977, Dipartimento del Commercio degli Stati Uniti.)

ISO9126

https://www.geeksforgeeks.org/iso-iec-9126-in-software-engineering/

Nella norma ISO 9126, uno sforzo simile è stato compiuto per definire un insieme di caratteristiche e sottocaratteristiche di qualità (vedi **Tabella 6.5**). Le loro definizioni sono riportate nelle **Tabelle 6.6** e **6.7**. Mentre i fattori ei criteri di qualità definiti in (McCall *et al.*, **1977**) sono fortemente correlati, lo schema ISO è gerarchico: ogni sottocaratteristica è correlata esattamente a una caratteristica.

Tabella 6.5 Caratteristiche qualitative e sottocaratteristiche della ISO 9126

Caratteristica	Sottocaratteristiche
Funzionalità	Adeguatezza Precisione Interoperabilità Sicurezza Conformità alla funzionalità
Affidabilità	Scadenza Tolleranza ai guasti Recuperabilità Conformità all'affidabilità
Usabilità	Comprensibilità Apprendibilità Operabilità Attrattiva Conformità all'usabilità
Efficienza	Comportamento temporale Utilizzo delle risorse Conformità all'efficienza
Manutenibilità	Analizzabilità Mutevolezza Stabilità Testabilità Conformità alla manutenibilità
Portabilità	Adattabilità Installabilità Coesistenza Sostituibilità Conformità alla portabilità

Le caratteristiche di qualità ISO si riferiscono strettamente ad un *prodotto software.* Le loro definizioni non catturano i problemi di qualità del *processo.* Ad esempio, la sicurezza può essere gestita in parte da disposizioni nel software e in parte da procedure adeguate. Solo il

primo è coperto dalla sottocaratteristica "sicurezza" dello schema ISO. Inoltre, le sottocaratteristiche riguardano aspetti di qualità *visibili* all'utente. La riutilizzabilità, ad esempio, non è inclusa nello schema ISO.

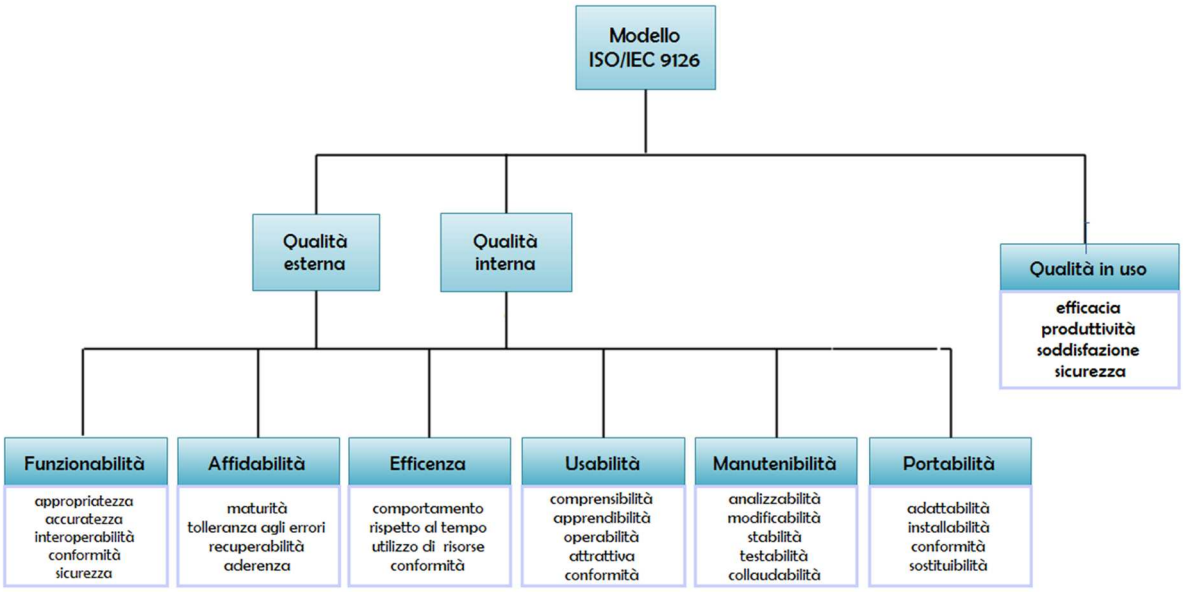
Tabella 6.6 Caratteristiche qualitative del modello di **qualità esterno ed interno** della ISO 9126

Funzionalità	La capacità del prodotto software di fornire funzioni che soddisfano le esigenze dichiarate e implicite quando il software viene utilizzato in condizioni specificate.
Affidabilità	La capacità del prodotto software di mantenere un determinato livello di prestazioni quando utilizzato in condizioni specificate.
Usabilità	La capacità del prodotto software di essere compreso, appreso, utilizzato e attraente per l'utente se utilizzato in condizioni specificate.
Efficienza	La capacità del prodotto software di fornire prestazioni adeguate, rispetto alla quantità di risorse utilizzate, alle condizioni stabilite.
Manutenibilità	La capacità del prodotto software di essere modificato. Le modifiche possono includere correzioni, miglioramenti o adattamenti del software ai cambiamenti dell'ambiente e dei requisiti e delle specifiche funzionali.
Portabilità	La capacità del prodotto software di essere trasferito da un ambiente all'altro.

(*Fonte:* Standard ISO 9126, *Caratteristiche e metriche di qualità del software*, © 2001. Riprodotto su autorizzazione di Netherlands Normalisatie-instituut.)

Le caratteristiche e le sottocaratteristiche dell'ISO, insieme a un'ampia serie di misure, costituiscono il *modello di qualità esterno e interno dell'ISO.* La qualità interna si riferisce al prodotto stesso, in definitiva al codice sorgente. La qualità esterna si riferisce alla qualità quando il software viene eseguito. Ad esempio, il numero medio di affermazioni in un metodo è una misura della qualità interna, mentre il numero di difetti riscontrati durante il test è una misura della qualità esterna.

In definitiva, l'utente è interessato alla *qualità in uso,* definita in (ISO 9126, **2001**) come *'vista dell'utente sulla qualità del prodotto software quando viene eseguito in un ambiente specifico e in un contesto d'uso specifico*'. Misura la misura in cui gli utenti possono raggiungere i propri obiettivi, piuttosto che semplici proprietà del software (vedere anche la **Sezione 6.3**). La qualità in uso è modellata in quattro caratteristiche: efficacia, produttività, sicurezza e soddisfazione. Le definizioni di queste caratteristiche della qualità d'uso sono riportate nella **tabella 6.8**.



Teoricamente, qualità interna, qualità esterna e qualità d'uso sono collegate: qualità interna indica qualità esterna che, a sua volta, indica qualità d'uso. In generale, soddisfare i criteri a un livello non è sufficiente per soddisfare i criteri al livello successivo. Ad esempio, la soddisfazione è in parte determinata da misure di qualità interne ed esterne, ma include anche l'atteggiamento dell'utente nei confronti del prodotto. Quest'ultimo deve essere misurato separatamente. Si noti che la qualità interna e la qualità esterna possono essere misurate direttamente. La qualità in uso può in generale essere misurata solo indirettamente.

Tabella 6.7 Sottocaratteristiche di qualità del modello di qualità esterno e interno della ISO 9126

Adeguatezza	La capacità del prodotto software di fornire un insieme appropriato di funzioni per attività specificate e obiettivi dell'utente.
Precisione	La capacità del prodotto software di fornire i risultati o gli effetti corretti o concordati con il grado di precisione necessario.
Interoperabilità	La capacità del prodotto software di interagire con uno o più sistemi specificati.
Sicurezza	La capacità del prodotto software di proteggere informazioni e dati in modo che persone o sistemi non autorizzati non possano leggerli o modificarli e che persone o sistemi autorizzati non ne abbiano accesso.
Conformità alla funzionalità	La capacità del prodotto software di aderire a standard, convenzioni o regolamenti in leggi e prescrizioni simili relative alla funzionalità.
Maturità	La capacità del prodotto software di evitare guasti a causa di guasti nel software.

Tolleranza ai guasti Fault tolerance	La capacità del prodotto software di mantenere un determinato livello di prestazioni in caso di guasti del software o di violazione dell'interfaccia specificata.
Recuperabilità	La capacità del prodotto software di ristabilire un determinato livello di prestazioni e recuperare i dati direttamente interessati in caso di guasto.
Conformità all'affidabilità	La capacità del prodotto software di aderire a standard, convenzioni o regolamenti relativi all'affidabilità.
Comprensibilità	La capacità del prodotto software di consentire all'utente di capire se il software è adatto e come può essere utilizzato per compiti e condizioni d'uso particolari.
Apprendibilità	La capacità del prodotto software di consentire all'utente di apprendere la sua applicazione.
Operabilità	La capacità del prodotto software di consentire all'utente di utilizzarlo e controllarlo.
Attrattività	La capacità del prodotto software di essere attraente per l'utente.
Conformità all'usabilità	La capacità del prodotto software di aderire a standard, convenzioni, guide di stile o regolamenti relativi all'usabilità.
Comportamento temporale	La capacità del prodotto software di fornire tempi di risposta e di elaborazione adeguati e velocità di trasmissione durante l'esecuzione della sua funzione, alle condizioni stabilite.
Utilizzo delle risorse	La capacità del prodotto software di utilizzare quantità e tipi di risorse appropriati quando il software svolge la sua funzione alle condizioni stabilite.
Conformità all'efficienza	La capacità del prodotto software di aderire a standard o convenzioni relative all'efficienza.
Analizzabilità	La capacità del prodotto software di essere diagnosticata per carenze o cause di guasti nel software, o per identificare le parti da modificare.
Mutevolezza	La capacità del prodotto software di consentire l'implementazione di una specifica modifica.
Stabilità	La capacità del prodotto software di evitare effetti imprevisti dalle modifiche del software.
Testabilità	La capacità del prodotto software di consentire la convalida del software modificato.
Conformità alla manutenibilità	La capacità del prodotto software di aderire a standard o convenzioni relative alla manutenibilità.
Adattabilità	La capacità del prodotto software di essere adattato per diversi ambienti specificati senza applicare azioni o mezzi diversi da quelli previsti a tale scopo per il software considerato.
Installabilità	La capacità del prodotto software di essere installato in un ambiente specifico.
Coesistenza	La capacità del prodotto software di coesistere con altri software indipendenti in un ambiente comune che condivide risorse comuni.

Sostituibilità	La capacità del prodotto software di essere utilizzato al posto di un altro prodotto software specificato per lo stesso scopo nello stesso ambiente.
Conformità alla portabilità	La capacità del prodotto software di aderire a standard o convenzioni relative alla portabilità.

(*Fonte:* Standard ISO 9126, *Caratteristiche e metriche di qualità del software*, © 2001. Riprodotto su autorizzazione di Netherlands Normalisatie-instituut.)

Tabella 6.8 Caratteristiche del modello di **qualità in uso** della ISO 9126

Efficacia	La capacità del prodotto software di consentire agli utenti di raggiungere determinati obiettivi con accuratezza e completezza in un determinato contesto di utilizzo.
Produttività	La capacità del prodotto software di consentire agli utenti di spendere quantità adeguate di risorse in relazione all'efficacia raggiunta in uno specifico contesto di utilizzo.
Sicurezza	La capacità del prodotto software di raggiungere livelli accettabili di rischio di danni a persone, aziende, software, proprietà o ambiente in uno specifico contesto di utilizzo.
Soddisfazione	La capacità del prodotto software di soddisfare gli utenti in uno specifico contesto di utilizzo.

(*Fonte:* Standard ISO 9126, *Caratteristiche e metriche di qualità del software*, © 2001. Riprodotto con il permesso di ISO.)

I fattori di qualità non sono indipendenti. Alcuni fattori avranno un impatto reciproco positivo, mentre altri lo faranno negativamente. Ad esempio, è più probabile che un programma affidabile sia corretto e viceversa. L'efficienza, d'altra parte, avrà generalmente un impatto negativo sulla maggior parte degli altri fattori di qualità. Ciò significa che dovremo fare dei compromessi tra i fattori di qualità. Se c'è un forte requisito per un fattore, potremmo dover rilassare altri fattori. Questi compromessi devono essere risolti in una fase iniziale. Un obiettivo importante della fase dell'architettura del software è portare questi fattori di qualità in primo piano e rendere espliciti i compromessi, in modo che le parti interessate sappiano a cosa servono (si veda **il Capitolo 11**). In questo modo, siamo in grado di costruire meglio le qualità desiderate, piuttosto che limitarci a valutarle a posteriori.

6.3 PROSPETTIVE SULLA QUALITÀ

- Quello che io (e tutti gli altri) intendo con la parola qualità non può essere scomposto in soggetti e predicati […] Se la qualità esiste in un oggetto, allora devi spiegare perché gli strumenti scientifici non sono in grado di rilevarla […] D'altra parte, se la qualità è soggettiva, esiste solo [agli occhi di] l'osservatore, allora questa Qualità è solo un nome di fantasia per qualunque cosa tu voglia […] La qualità non è oggettiva. Non risiede nel mondo materiale […] La qualità non è soggettiva. Non risiede

> semplicemente nella mente. Robert Pirsig, *Zen e l'arte della manutenzione della moto* (1974)

Gli utenti giudicheranno la qualità di un sistema software dal grado in cui li aiuta a portare a termine i compiti e dalla pura gioia che provano nell'usarlo. È probabile che il gestore di tali utenti giudichi la qualità dello stesso sistema in base ai suoi vantaggi. Questi vantaggi possono essere espressi in risparmi sui costi o in un servizio migliore e più rapido ai clienti.

Per un tester, le dimensioni di qualità prevalenti saranno il numero di difetti rilevati e rimossi, l'affidabilità misurata o la conformità alle specifiche. Per il programmatore di manutenzione, la qualità sarà correlata alla complessità del sistema, alla sua documentazione tecnica e così via.

Questi diversi punti di vista sono tutti validi. Sono anche difficili da conciliare. Garvin (**1984**) distingue cinque definizioni di qualità del software:

- Definizione trascendente
- Definizione basata sull'utente
- Definizione basata sul prodotto
- Definizione basata sulla produzione
- Definizione basata sul valore.

La qualità trascendente riguarda l'eccellenza innata. È il tipo di valutazione della qualità che di solito applichiamo ai romanzi. Possiamo considerare *Zen e l'arte della manutenzione della moto* un ottimo libro; possiamo provare a dare parole alla nostra ammirazione, ma queste parole di solito sono inadeguate. Il lettore esperto sviluppa gradualmente una buona sensazione per questo tipo di qualità. Allo stesso modo, l'esperto di ingegneria del software può sviluppare una buona sensibilità per le qualità trascendenti dei sistemi software.

La definizione di qualità basata sull'utente riguarda "idoneità all'uso" e si riferisce al grado in cui un sistema risponde alle esigenze dell'utente. È una nozione soggettiva. Poiché utenti diversi possono avere esigenze diverse, possono valutare la qualità di un sistema in modo piuttosto diverso. L'utente occasionale di un semplice pacchetto di elaborazione testi potrebbe essere abbastanza soddisfatto delle sue funzionalità e possibilità, mentre un informatico potrebbe essere piuttosto deluso. La situazione inversa può applicarsi a un sistema di composizione complesso come LaTeX.

Nella definizione basata sul prodotto, la qualità si riferisce agli attributi del software. Le differenze di qualità sono causate da differenze nei valori di tali attributi. La maggior parte della ricerca sulla qualità del software riguarda questo tipo di qualità. È anche alla base delle varie tassonomie di attributi di qualità discusse nella **Sezione 6.2** .

La definizione di qualità basata sulla produzione riguarda la conformità alle specifiche. È il tipo di definizione della qualità utilizzata durante i test di sistema, mentre la definizione basata sull'utente è prevalente durante i test di accettazione.

La definizione basata sul valore si occupa di costi e profitti. Si tratta di bilanciare tempo e costi da un lato e profitto dall'altro. Possiamo distinguere vari tipi di benefici, non tutti facilmente esprimibili in termini monetari (Simmons, **1996**):

- **Maggiore efficienza** I vantaggi sono attribuiti all'eliminazione o alla riduzione dei costi e le loro misure sono economiche.
- **Maggiore efficacia** Ciò si riflette principalmente in una migliore informazione per il processo decisionale. Può essere misurato in termini economici o attraverso indicatori chiave di performance, come un time to market ridotto.
- **Valore aggiunto** I vantaggi migliorano la posizione strategica dell'organizzazione, ad esempio attraverso una maggiore quota di mercato. Il contributo della componente informatica spesso non può essere isolato.
- **Prodotto commerciabile** Il sistema stesso può essere commerciabile o un prodotto commerciabile può essere identificato come un sottoprodotto dello sviluppo del sistema.
- **Infrastruttura IT aziendale Le** reti di comunicazione, gli ambienti di database e così via forniscono scarsi vantaggi di per sé, ma fungono da base per altri sistemi.

Gli sviluppatori di software tendono a concentrarsi sulle definizioni di qualità basate sul prodotto e sulla produzione. I requisiti di qualità che ne derivano possono essere espressi in termini quantificabili, come il numero di difetti riscontrati per mese uomo o il numero di punti di diramazione per modulo. Gli attributi di qualità discussi nella sezione precedente rientrano in queste categorie. Tali requisiti di qualità, tuttavia, non possono essere direttamente mappati sui punti di vista della qualità, piuttosto soggettivi, degli utenti, come l'"idoneità all'uso". Tuttavia, gli utenti e gli sviluppatori di software devono raggiungere un accordo sui requisiti di qualità da soddisfare.

Un modo per tentare di colmare questo divario è definire un linguaggio comune tra utenti e sviluppatori di software in cui possano essere espressi i requisiti di qualità. Questo approccio è adottato (Bass *et al.*, **2003**), dove i requisiti di qualità sono espressi in "scenari di attributi di qualità". **La Figura 6.3** fornisce un esempio di come un attributo di qualità può essere espresso in termini utente. Gli scenari di attributi di qualità hanno un ruolo non solo nello specificare i requisiti, ma anche nel verificare se questi requisiti saranno (saranno) soddisfatti. Ad esempio, gli scenari di attributi di qualità sono ampiamente utilizzati nelle valutazioni dell'architettura (si veda **il Capitolo 11**).

La qualità non è definita solo a livello dell'intero sistema. Nello sviluppo basato sui componenti, la qualità è definita a livello di un componente. Per i servizi, la qualità è definita a livello di servizio individuale. Anche l'ambiente del componente o servizio, ad esempio qualche altro componente o servizio, richiederà determinate qualità. Quindi, per componenti

e servizi, c'è un aspetto che richiede e fornisce un aspetto della qualità. Poiché un componente o un servizio generalmente non conosce il contesto in cui verrà inserito, è difficile decidere il livello di qualità "giusto". Si potrebbe quindi scegliere di offrire diversi livelli di qualità. Ad esempio, un servizio che gestisce lo streaming video può essere veloce con immagini di bassa qualità o lento con immagini di alta qualità. L'utente di quel servizio seleziona quindi il livello di qualità del servizio (QoS) appropriato. Gli aspetti relativi alla qualità dei componenti e dei servizi sono trattati rispettivamente nei **capitoli 18** e **19** .

Gli sviluppatori tendono ad avere una visione meccanicistica e orientata al prodotto sulla qualità, per cui la qualità è associata alle caratteristiche del prodotto. In questa visione, la qualità è definita guardando dal programma all'utente (facilità d'uso, accettabilità, ecc.). Per valutare la qualità dei sistemi utilizzati nelle organizzazioni, dobbiamo adottare anche una visione orientata al processo sulla qualità, in cui la qualità è definita guardando dall'utente al programma. Questo porta a nozioni come "adeguatezza" e "rilevanza". Ad esempio, un helpdesk con personale qualificato può contribuire notevolmente alla qualità di un sistema percepita dai suoi utenti, ma questo attributo di qualità generalmente non deriva da una visione della qualità basata sul prodotto.

Quality attribute	Usability
Source	End user
Stimulus	Learn system features
Artifact	System
Environment	At run time
Response	Learn tasks supplied by the system for new employees
Response measure	Days on the job
Test	90% successful completion of assigned tasks in employee test for the system, within twice the average time of an experienced user
Worst	1 to 7 days
Plan	Less than I day (to passing of test)
Best	Less than 2 hours

Figure 6.3 Uno scenario di attributi di qualità che può essere utilizzato sia dagli utenti che dagli sviluppatori

Una visione molto eclettica della qualità è presa in Total Quality Management (TQM). In TQM, la qualità si applica a ogni aspetto dell'organizzazione ed è perseguita da ogni dipendente di quell'organizzazione. TQM ha tre capisaldi:

- **Strategia di valore per il cliente:** La qualità è una combinazione di benefici derivati da un prodotto e sacrifici richiesti al cliente. Bisogna cercare il giusto equilibrio tra questi benefici e questi sacrifici. Lo stakeholder chiave in questo atto di bilanciamento è il cliente, piuttosto che il capo del cliente. L'atteggiamento non è 'Sappiamo cosa è meglio per il cliente', ma 'Prima determiniamo di cosa ha bisogno il cliente'.
- **Sistemi organizzativi:** I sistemi comprendono più di software e hardware. Anche altri materiali, esseri umani e pratiche di lavoro appartengono al sistema. Inoltre, i sistemi attraversano i confini di unità o dipartimenti. Nella vista TQM, i sistemi eliminano la complessità anziché le persone. In TQM, la cultura non è dominata dalle lotte di potere. Piuttosto, l'organizzazione si avvale dell'orgoglio dei dipendenti per l'artigianato. Le risorse umane sono considerate una risorsa critica e non un mero fattore di costo.
- **Miglioramento continuo** Un ambiente 'tradizionale' è reattivo: il miglioramento è innescato da un problema o dallo sviluppo di un nuovo prodotto. In TQM, la qualità è perseguita in modo proattivo. Gli errori non sono visti come fallimenti personali che

richiedono una punizione, ma come opportunità di apprendimento. Le prestazioni non vengono valutate in retrospettiva come buone o cattive, ma la variazione delle prestazioni viene analizzata statisticamente per comprendere le cause delle scarse prestazioni. L'autorità non è imposta da posizioni e regole, ma si guadagna comunicando una visione.

TQM sottolinea quindi il miglioramento piuttosto che la conformità. CMM (vedi **Sezione 6.6**) si basa su TQM e molte delle tecniche di ingegneria dei requisiti discusse nel **Capitolo 9** devono anche un tributo a TQM.

6.4 IL SISTEMA QUALITÀ

L'Organizzazione internazionale per la standardizzazione (ISO) ha sviluppato ISO 9000, una serie di standard per i sistemi di gestione della qualità (ISO 9000:2000, ISO 9001:2000 e ISO 9004:2000). La ISO 9000 fornisce i fondamenti e il vocabolario della serie di norme sui sistemi di qualità. ISO 9001 integra tre standard precedenti (etichettati ISO 9001, ISO 9002 e ISO 9003) e specifica i requisiti per un sistema di qualità per qualsiasi organizzazione che debba dimostrare la propria capacità di fornire prodotti che soddisfino i requisiti dei clienti. La ISO 9004 contiene linee guida per il miglioramento delle prestazioni. È applicabile dopo l'implementazione della ISO 9001.

ISO 9001 è uno standard generico che può essere applicato a qualsiasi prodotto. Un utile complemento per il software è la ISO/IEC 90003:2004, che contiene le linee guida per l'applicazione della ISO 9001 al software per computer. È uno standard congiunto di ISO e della Commissione elettrotecnica internazionale (IEC). Lo scopo della ISO/IEC 90003 è che "specifica i requisiti per un sistema di gestione della qualità in cui un'organizzazione:

- deve dimostrare la propria capacità di fornire costantemente un prodotto che soddisfi i requisiti normativi del cliente e applicabili, e
- mira a migliorare la soddisfazione del cliente attraverso l'applicazione efficace del sistema, compresi i processi per il miglioramento continuo del sistema e l'assicurazione della conformità al cliente e ai requisiti normativi applicabili.'

Lo standard è molto completo. Utilizza cinque prospettive da cui affrontare la gestione della qualità nell'ingegneria del software:

- La prospettiva sistemica si occupa dell'istituzione e della documentazione del sistema qualità stesso. Il sistema di qualità è costituito da una serie di processi, come quelli per lo sviluppo, il funzionamento e la manutenzione del software. Questi processi, e lo stesso sistema di qualità, devono essere adeguatamente documentati.
- La prospettiva del management si occupa della definizione e gestione delle politiche a supporto della qualità. Il sistema di gestione della qualità stesso deve essere sviluppato, implementato e regolarmente rivisto.
- La prospettiva delle risorse riguarda le risorse necessarie per implementare e migliorare il sistema di gestione della qualità, nonché per soddisfare i requisiti normativi e dei clienti. Le risorse includono il personale, le infrastrutture e l'ambiente di lavoro.

- La prospettiva del prodotto si occupa dei processi per creare effettivamente prodotti di qualità, come quelli relativi all'ingegneria dei requisiti, alla progettazione, al collaudo, alla produzione e all'assistenza. Questa prospettiva costituisce oltre il 60% dello standard.
- La prospettiva del miglioramento riguarda le attività di monitoraggio, misurazione e analisi per mantenere e migliorare la qualità.

Molte organizzazioni hanno cercato di ottenere la registrazione ISO 9000. Il tempo e il costo necessari dipendono da quanto i processi attuali si discostano dagli standard ISO. Se l'attuale sistema di qualità non è già vicino alla conformità, la registrazione ISO potrebbe richiedere almeno un anno. La registrazione ISO viene concessa quando un ente terzo accreditato valuta il sistema di qualità e conclude che è conforme allo standard ISO. La nuova registrazione è richiesta ogni tre anni e gli audit di sorveglianza sono richiesti ogni sei mesi. La registrazione ISO è quindi un affare abbastanza drastico e costoso, dopo di che non puoi certo sederti ma devi tenere l'organizzazione all'erta.

Poiché i progetti di sviluppo software hanno alcune caratteristiche piuttosto peculiari (frequenti cambiamenti nei requisiti durante il processo di sviluppo e la natura invisibile del prodotto durante il suo sviluppo), sono necessarie procedure di garanzia della qualità adeguate allo sviluppo del software. Questo è l'argomento della **Sezione 6.5**.

6.5 SOFTWARE QUALITY ASSURANCE - GARANZIA DELLA QUALITÀ DEL SOFTWARE

Lo scopo della garanzia della qualità del software (SQA) è assicurarsi che il lavoro venga svolto nel modo in cui dovrebbe essere svolto. In particolare, gli obiettivi di SQA (Humphrey, **1989**) sono:

- migliorare la qualità del software monitorando adeguatamente il software e il suo processo di sviluppo;
- assicurare il pieno rispetto degli standard e delle procedure stabilite per il software e il processo di sviluppo;
- per garantire che eventuali inadeguatezze nel prodotto, nel processo o negli standard siano portate all'attenzione della direzione in modo che queste inadeguatezze possano essere risolte.

Si noti che le persone SQA stesse non sono responsabili della produzione di prodotti di qualità. Il loro compito è quello di rivedere e controllare, e di fornire al progetto e alla direzione i risultati di queste revisioni e audit.

Esistono potenziali conflitti di interesse tra l'organizzazione SQA e l'organizzazione di sviluppo. L'organizzazione di sviluppo potrebbe dover affrontare delle scadenze e voler spedire un prodotto, mentre il personale SQA ha rivelato seri problemi di qualità e desidera differire la spedizione. In tali casi, dovrebbe prevalere il parere dell'organizzazione SQA. Affinché l'SQA sia efficace, devono essere soddisfatti alcuni prerequisiti:

- È essenziale garantire l'impegno del top management, in modo che i suggerimenti dell'organizzazione SQA possano essere applicati. In caso contrario, l'SQA diventa presto un'imbottitura costosa e un semplice fastidio per l'organizzazione di sviluppo.
- L'organizzazione SQA dovrebbe essere indipendente dall'organizzazione di sviluppo. Anche la sua linea di segnalazione dovrebbe essere indipendente.
- L'organizzazione SQA dovrebbe essere dotata di personale tecnicamente competente e giudizioso. Devono collaborare con l'organizzazione di sviluppo. Se le due organizzazioni operano come avversari, SQA non può essere efficace. Dobbiamo renderci conto che, nel lungo periodo, gli obiettivi dell'organizzazione SQA e dell'organizzazione di sviluppo sono gli stessi: la produzione di prodotti di alta qualità.

Le attività di revisione e audit e gli standard e le procedure da seguire sono descritti in un piano di garanzia della qualità del software.

Lo standard IEEE 730 offre una struttura per i contenuti di un piano di garanzia della qualità per lo sviluppo del software (IEEE 730, **1998**). La **Tabella 6.9** elenca il contenuto di tale documento. Lo standard IEEE 730 si applica allo sviluppo e alla manutenzione di software critici. Per il software non critico, può essere utilizzato un sottoinsieme dei requisiti.

Tabella 6.9 Componenti principali dello standard IEEE 730

1.	Scopo
2.	Documenti di riferimento
3.	Gestione
4.	Documentazione
5.	Standard, pratiche, convenzioni e metriche
6.	Revisioni e audit
7.	Test
8.	Segnalazione dei problemi e azioni correttive
9.	Strumenti, tecniche e metodologie
10.	Controllo del codice
11.	Controllo dei media

12. Controllo del fornitore
13. Raccolta, manutenzione e conservazione dei record
14. Addestramento
15. Gestione del rischio

Lo standard IEEE 983 (IEEE983, **1986**) è un utile complemento allo standard 730. Lo standard IEEE 983 offre ulteriori linee guida sui contenuti di un piano di garanzia della qualità e sulla sua implementazione, valutazione e modifica.

Un piano di garanzia della qualità del software descrive come valutare la qualità del software. Alcuni fattori di qualità possono essere determinati oggettivamente. La maggior parte dei fattori attualmente può essere determinata solo soggettivamente. Molto spesso, quindi, cercheremo di valutare la qualità leggendo documenti, ispezioni, procedure dettagliate e revisioni tra pari. In un certo numero di casi, possiamo utilizzare con profitto strumenti durante l'assicurazione della qualità, in particolare per l'analisi statica e dinamica del codice del programma. Le tecniche effettive da applicare qui sono discusse nel **Capitolo 13** .

6.6 IL CAPABILITY MATURITY MODEL (CMM)

Considera il seguente corso di eventi in un ipotetico progetto di sviluppo software. Un'organizzazione deve sviluppare un sistema di automazione delle biblioteche distribuito. Un computer centralizzato ospita sia il software che il database. Diverse librerie locali sono collegate alla macchina centrale tramite un'interfaccia basata sul Web. L'organizzazione ha una certa esperienza con l'automazione delle biblioteche, anche se solo con sistemi autonomi.

Nel corso del progetto si manifestano una serie di problemi. All'inizio sembrano disconnessi e non attivano la gestione degli allarmi. Si scopre che l'analisi dei requisiti non è stata così approfondita. I requisiti locali risultano diversi su alcuni punti minori. Sebbene le prime deviazioni di questo tipo possano essere gestite abbastanza facilmente, tenere traccia di tutte le richieste di modifica diventa un vero problema dopo un po'. Quando una parte del sistema è stata realizzata, il team inizia a testare l'interfaccia Web, ma risulta essere troppo complessa e dispendiosa in termini di tempo.

Alla fine il progetto entra in crisi. La direzione non ha mezzi adeguati per gestire la situazione. Cerca di ridurre sia la funzionalità che la qualità in modo un po' casuale. Alla fine, un sistema piuttosto insoddisfacente viene consegnato con due mesi di ritardo. Durante la successiva fase di manutenzione, alcuni problemi vengono risolti, ma il sistema non diventa mai un vero successo.

Sebbene la descrizione di cui sopra sia ipotetica, non è poi così irrealistica. Molte organizzazioni hanno un controllo insufficiente sul processo di sviluppo del software. Se un progetto ha problemi, di solito viene scoperto piuttosto tardi e l'organizzazione può reagire solo in modo un po' caotico. Il più delle volte, la velocità viene confusa con il progresso.

Un passo importante nel tentativo di affrontare questi problemi è rendersi conto che il processo di sviluppo del software può effettivamente essere controllato, misurato e migliorato. Per valutare il processo di miglioramento del processo di sviluppo del software, Watts Humphrey ha sviluppato un framework di maturità del software che si è evoluto nel *Capability Maturity Model* (CMM). Questo quadro è un tributo al Total Quality Management (TQM), che a sua volta si basa sui principi del controllo statistico della qualità formulati da Walter Shewart negli anni '30 e ulteriormente sviluppati da W. Edwards Deming e Joseph Juran negli anni '80. In origine, esistevano modelli CMM separati per l'ingegneria del software, l'ingegneria dei sistemi e molti altri. Questi sono stati ora integrati e portano l'etichetta CMMI.[2] CMM e CMMI sono stati sviluppati presso il Software Engineering Institute (SEI) della Carnegie Mellon University. La versione qui descritta è CMMI versione 1.2 per l'ingegneria del software e l'ingegneria dei sistemi (CMMI Product Team, **2006**).

In CMM (e CMMI), il processo software è caratterizzato come uno dei cinque livelli di **maturità,** livelli evolutivi verso il raggiungimento di un processo software maturo. Per raggiungere un certo livello di maturità, è necessario disporre di un certo numero di **aree di processo.** Queste aree di processo indicano questioni importanti che devono essere affrontate per raggiungere quel livello. Nel loro insieme, le aree di processo di un livello raggiungono la serie di obiettivi per quel livello. La **Figura 6.4** elenca i livelli di maturità e le aree di processo associate di CMMI.

I livelli di maturità di CMMI possono essere caratterizzati come segue:

1. **Livello iniziale** A questo livello, l'organizzazione opera senza procedure formalizzate, piani di progetto o stime dei costi. Gli strumenti non sono adeguatamente integrati. Molti problemi vengono trascurati o dimenticati e la manutenzione pone problemi reali. Lo sviluppo di software a questo livello può essere caratterizzato come ad hoc. Le prestazioni possono essere migliorate istituendo controlli di base sulla gestione del progetto:
    - **La gestione dei requisiti** implica la definizione e il mantenimento di un accordo con il cliente sui requisiti del sistema. Poiché i requisiti cambiano inevitabilmente, il controllo e la documentazione di questi requisiti è importante.
    - La **pianificazione del progetto** implica l'elaborazione di piani per l'esecuzione e la gestione del progetto. Per poter eseguire qualsiasi pianificazione, è necessaria una dichiarazione approvata del lavoro da svolgere. Da questa dichiarazione di lavoro, vengono determinate le stime delle dimensioni del progetto, le risorse necessarie e il programma e vengono identificati i rischi per il progetto. I risultati sono documentati nel piano di progetto. Questo piano viene utilizzato per gestire il progetto; viene aggiornato quando necessario.

- **Il monitoraggio e il controllo del progetto** riguarda la visibilità dei progressi effettivi. I risultati intermedi devono essere rivisti e tracciati rispetto al piano del progetto. Quando necessario, il piano del progetto deve essere riallineato alla realtà.
- **Gestione accordi fornitori.** Ove applicabile, il lavoro svolto dai fornitori deve essere gestito: è necessario pianificare la loro parte di lavoro e monitorare l'andamento della loro parte di lavoro.
- **La misurazione e l'analisi** si occupano di assicurarsi che le misurazioni vengano eseguite e che i loro risultati vengano utilizzati. In primo luogo, vengono stabiliti gli obiettivi per la misurazione e il modo in cui le misure dovrebbero essere raccolte, archiviate e analizzate. Successivamente, devono essere implementate la raccolta, l'archiviazione e l'analisi dei dati. Infine, i risultati vengono utilizzati per il processo decisionale e vengono intraprese azioni correttive ove necessario.
- **La garanzia della qualità dei processi e dei prodotti** implica la revisione e l'audit di prodotti e processi per convalidarne la conformità agli standard e alle procedure concordate.
- La **gestione della configurazione** si occupa di stabilire e mantenere l'integrità di tutti gli elementi di lavoro durante l'intero ciclo di vita del progetto. Ciò comporta l'identificazione degli elementi di configurazione e delle linee di base e l'istituzione di procedure per controllare le modifiche agli stessi.

2. **Ripetibile** La principale differenza tra il livello di processo iniziale e il livello di processo ripetibile è che c'è il controllo sul modo in cui vengono stabiliti i piani e gli impegni. Grazie a precedenti esperienze nello svolgimento di lavori simili, l'organizzazione ha ottenuto il controllo su costi, programmi e richieste di modifica e i successi precedenti possono essere ripetuti. L'introduzione di nuovi strumenti, lo sviluppo di una nuova tipologia di prodotto e importanti cambiamenti organizzativi, tuttavia, rappresentano ancora rischi importanti a questo livello. Le aree di processo necessarie per passare al livello successivo mirano a standardizzare il processo software *attraverso* i progetti dell'organizzazione:
    - **Lo sviluppo dei requisiti** implica la produzione e l'analisi dei requisiti. I requisiti devono essere sollecitati, analizzati, convalidati e comunicati alle parti interessate appropriate. Lo sviluppo dei requisiti non è un'attività one-shot. Piuttosto, i requisiti vengono identificati e perfezionati durante l'intero ciclo di vita.
    - **La soluzione tecnica** riguarda la progettazione e l'implementazione. Le decisioni riguardanti l'architettura, lo sviluppo personalizzato rispetto a una soluzione standard e i problemi di modularizzazione sono ingredienti tipici di questa area di processo.
    - **L'integrazione di prodotto** riguarda l'assemblaggio di un prodotto completo dai suoi componenti. Questa può essere una fase dopo che tutti i componenti sono stati sviluppati o può procedere in modo incrementale. Un elemento importante dell'integrazione del prodotto è la gestione delle interfacce, per assicurarsi che i componenti si incastrino correttamente.
    - La **verifica** si occupa di garantire che il prodotto soddisfi i suoi requisiti. Le revisioni tra pari sono un mezzo efficace per il rilevamento e la rimozione precoci dei difetti. Le

revisioni tra pari, come procedure dettagliate e ispezioni, sono pratiche in cui i colleghi cercano di identificare gli errori e le aree in cui sono necessarie modifiche.

- **La convalida** si occupa di stabilire che il prodotto soddisfa l'uso previsto. Per quanto possibile, le attività di convalida dovrebbero essere svolte nell'ambiente in cui il prodotto è destinato ad essere utilizzato.
- **Il focus del processo organizzativo** riguarda il miglioramento del processo organizzativo. Le misurazioni, le lezioni apprese, le autopsie del progetto e i rapporti di valutazione del prodotto sono fonti di informazioni tipiche per guidare le attività di miglioramento. La responsabilità di guidare e implementare queste attività è generalmente assegnata a un gruppo di processi. In questo modo, il miglioramento delle capacità di processo dell'organizzazione diventa una responsabilità dell'organizzazione nel suo insieme, piuttosto che del singolo project manager.
- **La definizione del processo organizzativo** riguarda lo sviluppo e il mantenimento di una serie di risorse di processo software, come elementi di processo, modelli del ciclo di vita e linee guida per la personalizzazione di un modello di processo. Ogni progetto utilizza un processo costruito a partire da queste risorse di processo.
- **La formazione organizzativa** riguarda la creazione di un programma di formazione che sviluppi le competenze e le conoscenze necessarie degli individui per svolgere i propri ruoli. I bisogni formativi sono identificati a livello di organizzazione, progetto e individuo. Anche il soddisfacimento di questi bisogni è affrontato.
- **La gestione integrata del progetto** implica lo sviluppo di un processo software specifico del progetto dall'insieme di processi standard dell'organizzazione, nonché l'effettiva gestione del progetto utilizzando il processo su misura. Poiché i processi software di diversi progetti hanno un predecessore comune, i progetti possono ora condividere dati e lezioni apprese.
- La **gestione del rischio** riguarda l'identificazione di potenziali problemi, in modo da poter intraprendere azioni tempestive per mitigare gli effetti negativi.
- **L'analisi e la risoluzione delle decisioni** riguarda la definizione di linee guida su quali problemi dovrebbero essere sottoposti a processi di valutazione formale e l'applicazione di tali processi formali. La selezione di componenti commerciali off-the-shelf (COTS) e le revisioni dell'architettura sono aree in cui potrebbero essere applicati processi di valutazione formale.

3. **Definito** A livello di processo definito, è in atto una serie di processi standard per lo sviluppo e la manutenzione del software. L'organizzazione ha raggiunto solide basi e ora può iniziare a esaminare i propri processi e decidere come migliorarli. Ci sono passaggi importanti per passare al livello successivo:
   - **Le prestazioni dei processi organizzativi** hanno lo scopo di stabilire e mantenere una comprensione quantitativa delle prestazioni dell'insieme dei processi standard. I singoli progetti vengono misurati e confrontati con i risultati attesi, come documentato in una baseline. Le informazioni non vengono utilizzate solo per valutare un progetto, ma anche per gestirlo quantitativamente.
   - La **gestione quantitativa del progetto** prevede la definizione di obiettivi di prestazione, la misurazione delle prestazioni del processo, l'analisi di tali misurazioni

e l'applicazione degli opportuni adeguamenti al processo per portarlo in linea con i limiti definiti. Esiste, quindi, un programma di misurazione a livello di organizzazione e i risultati di esso vengono utilizzati per migliorare continuamente il processo. Un esempio di misura di processo è il numero di righe di codice riviste all'ora.

4. **Gestione quantitativa** A livello di processo gestito quantitativamente, i dati quantitativi vengono raccolti e analizzati su base di routine. Tutto è sotto controllo e l'attenzione può quindi passare dall'essere reattivi (cosa succede al progetto attuale?) all'essere proattivi (cosa possiamo fare per migliorare i progetti futuri?). L'attenzione si sposta sulle opportunità di miglioramento continuo:
    - **L'innovazione e l'implementazione organizzative** riguardano l'identificazione e l'implementazione di miglioramenti. I miglioramenti tecnici riguardano le nuove tecnologie e la loro ordinata transizione nell'organizzazione. I miglioramenti di processo riguardano la modifica del processo al fine di migliorare la qualità dei prodotti e la produttività dell'organizzazione di sviluppo software e di ridurre il tempo necessario per sviluppare i prodotti.
    - **L'analisi e la risoluzione causali** riguarda l'identificazione delle cause comuni dei difetti e la prevenzione del loro ripetersi.
5. **Ottimizzazione** Al livello finale, di ottimizzazione, è stata raggiunta una base stabile dalla quale è possibile apportare ulteriori miglioramenti. Il passaggio al livello di processo di ottimizzazione è un cambio di paradigma. Mentre l'attenzione agli altri livelli è focalizzata sui modi per migliorare il prodotto, l'enfasi al livello di ottimizzazione si è spostata dal prodotto al processo. I dati raccolti possono ora essere utilizzati per migliorare il processo di sviluppo del software stesso.

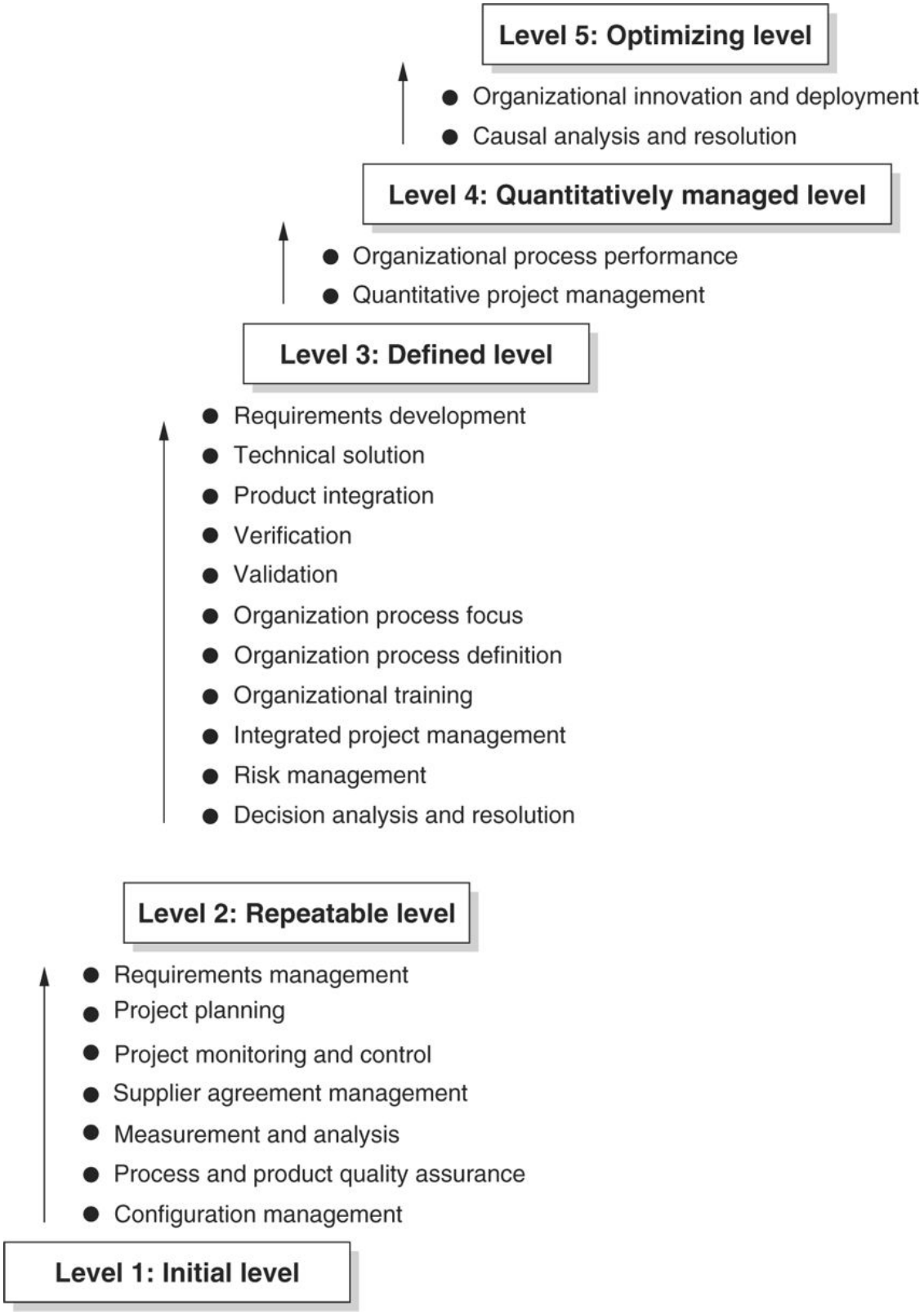


Figure 6.4 Livelli di maturità e aree di processo associate di CMMI

Nel 1989, Humphrey ha studiato lo stato della pratica dell'ingegneria del software rispetto al CMM (Humphrey *et al.*, **1989**). Sebbene questo studio abbia interessato la comunità del software del Dipartimento della Difesa (DoD), non ci sono molte ragioni per aspettarsi che la situazione fosse molto più rosea in altri ambienti. Secondo le sue scoperte, la pratica dell'ingegneria del software a quel tempo era in gran parte al livello iniziale. C'erano alcune organizzazioni che operavano a livello ripetibile e alcuni progetti che operavano a livello definito. Nessuna organizzazione o progetto operato a livelli gestiti o ottimizzanti.

Negli anni successivi sono successe molte cose. Molte organizzazioni hanno avviato un programma di miglioramento dei processi software (SPI) per raggiungere un livello di maturità più elevato. La maggior parte di questi programmi di miglioramento riguarda il passaggio al livello ripetibile o definito. Il numero di organizzazioni a questi livelli è notevolmente aumentato dal 1989. Ci sono ancora poche organizzazioni o progetti ai livelli gestiti o di ottimizzazione.

I resoconti dell'esperienza pratica mostrano che ci vogliono circa due anni per salire di livello. Il costo varia da $500 a $2000 per dipendente all'anno. I vantaggi, tuttavia, sembrano superare facilmente il costo. Diverse aziende hanno riportato un ritorno sull'investimento di almeno 5 a 1: ogni dollaro investito in un programma di miglioramento dei processi ha comportato un risparmio sui costi di almeno cinque dollari.

6.6.1 Personal Software Process

Per soddisfare le esigenze delle piccole imprese e dei piccoli team di progetto, il Software Engineering Institute ha sviluppato il Personal Software Process (PSP), un processo di auto-miglioramento progettato per aiutare le persone a migliorare il proprio modo di lavorare. Come il CMM, il PSP distingue tra diversi livelli di maturità. Il primo passo in PSP è stabilire alcune misurazioni di base, come il tempo di sviluppo e i difetti rilevati, utilizzando semplici moduli per raccogliere questi dati. Al livello successivo, i dati vengono utilizzati per stimare il tempo e la qualità. A livelli ancora più elevati, i dati personali vengono utilizzati per *migliorare* le prestazioni dell'individuo.

I principi base della CMM e della PSP sono quindi molto simili: conoscere il proprio processo, misurare le proprie prestazioni e basare le proprie azioni di miglioramento sull'analisi dei dati raccolti.

6.6.2 BOOTSTRAP e SPICE

BOOTSTRAP e SPICE sono altri due modelli di maturità simili a CMM. BOOTSTRAP utilizza un rating di maturità separato per ciascuna delle sue pratiche. Una delle caratteristiche interessanti di BOOTSTRAP è che tutti i risultati della valutazione vengono raccolti in un database, consentendo così a un'organizzazione di posizionarsi confrontando i propri punteggi con quelli di organizzazioni simili.

SPICE è un'iniziativa internazionale ed è diventata uno standard internazionale (ISO/IEC 15504). SPICE è l'acronimo di Software Process Improvement e Capability determination. SPICE distingue diverse categorie di processi, come il processo di gestione, il processo cliente-fornitore e il processo di progettazione. Il livello di capacità (maturità) è determinato per ogni categoria di processo e ogni processo. Come BOOTSTRAP, SPICE si traduce quindi in un profilo di maturità. La metodologia SPICE pone un forte accento sul modo in cui vengono eseguite le valutazioni dei processi.

6.6.3 Alcune note critiche

- Le organizzazioni di sviluppo software esistono per sviluppare software piuttosto che processi. Fayad (**1997**)

L'enorme attenzione delle organizzazioni nell'ottenere la certificazione CMM o ISO 9000 nasconde il pericolo che l'attenzione si sposti dallo sviluppo di software allo sviluppo di processi. Un'organizzazione certificata non garantisce la qualità del software sviluppato sotto di essa. Un processo di sviluppo maturo non è un proiettile d'argento; un certificato incorniciato sicuramente non lo è.

CMM sembra più appropriato per le aziende molto grandi. È dubbio che le piccole imprese possano permettersi il tempo e il denaro richiesti da un programma di miglioramento dei processi, come auspicato da CMM. È anche dubbio che possano permettersi di implementare alcune delle aree di processo, come l'area "focus sul processo organizzativo", che richiede la creazione di un gruppo di processi organizzativi. Sebbene PSP possa alleviare parte di questa critica, il PSP non ha lo stesso status del CMM.

CMM è incentrato sulla disciplina: processi di lavoro strutturati, piani rigorosi e standardizzazione. Questo si adatta meglio alle aziende più grandi rispetto a quelle piccole. Si adatta meglio anche alle attività che si prestano a un approccio rigoroso, come la gestione della configurazione e il test. L'analisi dei requisiti e la progettazione richiedono una certa quantità di creatività e un approccio CMM puro può avere un effetto soffocante. Qui emerge anche la dicotomia notata nel **Capitolo 1** tra gli aspetti di fabbrica e di artigianato dell'ingegneria del software.

I livelli di maturità originari di CMM costituiscono una scala di cinque punti piuttosto grezza. Se la valutazione di un'organizzazione di livello 2 rivela che non soddisfa i criteri di livello 3 solo su una questione, il verdetto è piuttosto duro: l'organizzazione rimane semplicemente al livello 2. Ciò potrebbe non migliorare il morale dopo due anni di duro lavoro e investimenti significativi. Per prima cosa, questa implicazione delle valutazioni della maturità pone requisiti elevati alla loro affidabilità. La valutazione piuttosto grossolana delle organizzazioni su una scala di cinque punti può avere altre conseguenze di vasta portata. Il governo degli Stati Uniti richiede la certificazione di livello 3 per qualificarsi per i contratti. Ciò implica che le organizzazioni di livello 1 e di livello 2 stiano ottenendo prestazioni al di sotto degli standard? Se la certificazione di livello 3 è tutto ciò che conta, vale la pena puntare al livello 4 o 5?

I livelli originali di CMM sono come un quadro strumenti di un aeroplano con un indicatore, che può visualizzare solo pochi valori discreti; fornisce al pilota pochissime informazioni. Si può anche prevedere un "quadro strumenti" della maturità del software con molti indicatori, ognuno dei quali mostra molti dettagli. BOOTSTRAP e SPICE sono framework che si traducono in un profilo di maturità piuttosto che in un singolo punteggio. Lo stesso vale per CMMI, che è disponibile in due varianti: un **modello a fasi** che, come la CMM originale, ha solo cinque livelli di maturità e un **modello continuo** in cui il miglioramento dei processi viene effettuato in base all'area di processo.

Il miglioramento dei processi CMM e software si concentra sull'eccellenza operativa: l'organizzazione fornirà i prodotti migliori se i suoi processi sono ottimali. Un'organizzazione può, tuttavia, avere anche altri obiettivi, come una maggiore intimità con il cliente, una quota di mercato maggiore o l'innovazione del prodotto. Tali obiettivi richiedono obiettivi SPI diversi da quelli a cui siamo abituati (Rifkin, **2001**).

6.7 PER INIZIARE

Nelle sezioni precedenti, abbiamo discusso vari modi per rivedere la qualità di un prodotto software e il processo di sviluppo associato. La stessa organizzazione di sviluppo dovrebbe perseguire attivamente la produzione di prodotti di qualità, stabilendo obiettivi di qualità, valutando le proprie prestazioni e intraprendendo azioni per migliorare il processo di sviluppo.

Ciò richiede la comprensione delle possibili inadeguatezze nel processo di sviluppo e delle possibili cause. Tale comprensione deve essere ottenuta attraverso la raccolta di dati sia sul processo che sui prodotti risultanti e una corretta interpretazione di tali numeri. È piuttosto facile raccogliere enormi quantità di dati e applicarvi vari tipi di tecniche di adattamento della curva. Per poter interpretare correttamente le tendenze osservate, queste dovrebbero essere supportate da solide ipotesi.

Un esempio, certamente ridicolo, è riportato nella **Tabella 6.10** . I numeri in questa tabella indicano che le mucche nere producono più latte delle mucche bianche. Un'interpretazione piuttosto ingenua è che la produttività può essere migliorata in modo significativo ridipingendo tutte le vacche bianche.

Tabella 6.10 Relazione tra il colore delle vacche e la produzione media di latte

Colore	Produzione media
Bianco	10
Nero	40

Sebbene l'esempio stesso sia ridicolo, la sua controparte nell'ingegneria del software non è poi così inverosimile. Molti studi, ad esempio, hanno cercato di determinare una relazione tra i numeri che indicano la complessità dei componenti software e la qualità di tali componenti. Molti di questi studi hanno trovato una correlazione positiva tra tali dati di complessità e, ad esempio, il numero di difetti riscontrati durante i test. Un'interpretazione diretta di questi risultati è quindi quella di imporre un limite superiore alla complessità consentita per ciascun componente. Tuttavia, potrebbero esserci buone ragioni per alcuni componenti che hanno un'elevata complessità. Ad esempio, Redmond e Ah-Chuen (**1990**) hanno studiato le metriche di complessità di un gran numero di moduli del sistema operativo MINIX. Alcuni di questi, come un modulo che gestisce le sequenze di caratteri di escape dalla tastiera, erano considerati giustamente complessi. Gli esperti hanno ritenuto non giustificata un'ulteriore scomposizione di questi moduli. Mettere un semplice limite superiore al valore consentito di alcune metriche di complessità è un approccio troppo semplicistico.

Un'organizzazione deve scoprire le proprie opportunità di miglioramento dei processi. Il modo preferito per farlo è seguire un approccio graduale ed evolutivo in cui possono essere identificati i seguenti passaggi:

- 1. Formula ipotesi.
- 2. Seleziona con attenzione le metriche appropriate.
- 3. Raccogliere dati.
- 4. Interpreta i dati.
- 5. Avviare azioni di miglioramento.

Questi passaggi vengono ripetuti, in modo che l'effetto delle azioni sia convalidato e vengano formulate ulteriori ipotesi. Seguendo questo approccio, la ricerca della qualità permeerà la tua organizzazione, che in seguito ne trarrà vantaggio.

Un esempio di questo approccio è discusso in (Genuchten, **1991**). Descrive uno studio empirico delle ragioni del ritardo nello sviluppo del software. Lo studio ha riguardato sei progetti di sviluppo da un dipartimento. L'attenzione si è concentrata sulla raccolta di dati relativi al tempo e all'impegno, come le differenze tra un piano e la realtà. A tale scopo è stato utilizzato un modulo di raccolta dati di una pagina (cfr. **tabella 6.11**).

Tabella 6.11 Scheda attività per un'attività

	Pianificato	Effettivo	Differenza	Motivo
Sforzo	–	–	–	–
Data di inizio	–	–	–	–
Data di fine	–	–	–	–

Durata	–	–	–	–

Sono state individuate circa 30 ragioni per il ritardo. Questi sono stati classificati in sei categorie dopo una discussione con i leader del progetto e finalizzati dopo uno studio pilota. I motivi del ritardo sono risultati specifici dell'ambiente.

Un totale di 160 attività sono state studiate dalla metà del 1988 alla metà del 1989. Circa il 50% delle attività ha superato il piano di oltre il 10%. Il confronto tra i dati pianificati e quelli effettivi ha mostrato che le differenze relative sono aumentate verso la fine dei progetti. Si è riscontrato che una delle ragioni principali della differenza tra piano e realtà era "più tempo dedicato ad altro lavoro del previsto". I risultati sono stati interpretati durante un incontro con i capi progetto e il responsabile del dipartimento. La discussione ha confermato e quantificato alcune impressioni esistenti. Per alcuni, la discussione ha fornito nuove informazioni. Ha dimostrato che le azioni di manutenzione interrompevano costantemente il lavoro di sviluppo. L'incontro prevedeva una discussione sulle possibili azioni di miglioramento. È stato deciso di programmare la manutenzione il più possibile in "settimane di manutenzione" e includerla nei piani trimestrali. È stato avviato un altro studio di analisi per ottenere ulteriori approfondimenti sulle attività di manutenzione.

Questo studio fornisce una serie di spunti utili, alcuni dei quali rafforzano le affermazioni fatte in precedenza:

- Il principio del "circuito chiuso" afferma che i sistemi informativi dovrebbero essere progettati in modo tale che coloro che forniscono input al sistema siano anche i principali utenti del suo output. L'applicazione di questo principio si traduce in un feedback al fornitore di dati, che è quindi costretto a fornire un input accurato. Impedisce inoltre agli utenti di chiedere più del necessario. Nell'esempio sopra, i dati sono stati raccolti e analizzati dai leader del progetto. Il risultato è stato riferito agli stessi leader di progetto e utilizzato come punto di partenza per ulteriori azioni.
- La raccolta dei dati locali dovrebbe essere per uso locale. I dati raccolti possono variare considerevolmente tra i reparti. I dati vengono utilizzati al meglio per ottenere informazioni dettagliate sulle prestazioni del dipartimento in cui vengono raccolti i dati. L'uso in un altro reparto non ha molto senso.
- L'obiettivo dovrebbe essere il miglioramento continuo. Lo sforzo di raccolta dei dati era volto a individuare le carenze percepite nel processo di sviluppo del software. Ha rivelato le cause di queste carenze e ha fornito un'opportunità di miglioramento. La domanda non è "chi ha ragione e chi ha torto", ma piuttosto "come possiamo evitare che ciò accada di nuovo nei progetti futuri".
- Lo studio non ha comportato una massiccia raccolta di dati. Sono state utilizzate semplici schede tecniche, insieme a definizioni univoche del significato delle varie metriche. L'approccio è incrementale, con lo studio che offre l'opportunità di piccoli miglioramenti e mostra la strada per il prossimo studio.

6.8 RIEPILOGO

In questo capitolo abbiamo prestato ampia attenzione alla nozione di qualità. La qualità del software non è gratuita. Deve essere attivamente perseguito. L'uso di un modello ben definito del processo di sviluppo del software e di buone tecniche di analisi, progettazione e implementazione sono un prerequisito. Tuttavia, anche la qualità deve essere controllata e gestita. Per poterlo fare, deve essere definito rigorosamente. Questo non è senza problemi, come abbiamo visto nelle **Sezioni 6.2** e **6.3**. Esistono numerose tassonomie di attributi di qualità. Per ciascuno di questi attributi, abbiamo bisogno di una definizione precisa, insieme a una metrica che possa essere utilizzata per stabilire gli obiettivi di qualità e per verificare che questi obiettivi di qualità siano effettivamente soddisfatti. La maggior parte degli attributi di qualità si riferiscono ad aspetti che interessano principalmente agli sviluppatori di software. Queste visioni della qualità orientate agli ingegneri sono difficili da conciliare con gli aspetti di "idoneità all'uso" orientati all'utente.

Per la maggior parte degli attributi di qualità, la relazione tra ciò che viene effettivamente misurato (struttura del modulo, difetti riscontrati, ecc.) e l'attributo che ci interessa non è sufficientemente supportata da un'ipotesi valida. Ad esempio, sebbene i programmi con un gran numero di punti decisionali siano spesso complessi, esistono controesempi che mostrano che il numero di decisioni (essenzialmente, la complessità ciclomatica di McCabe) non è una buona misura della complessità del programma. La questione delle metriche del software e dei problemi associati è trattata ulteriormente nel **Capitolo 12** .

I principali standard per i sistemi di qualità sono stati definiti da ISO e IEEE. Questi standard forniscono linee guida dettagliate per quanto riguarda la gestione della qualità. La garanzia della qualità di per sé non garantisce prodotti di qualità. Deve essere integrato da un programma di qualità all'interno dell'organizzazione di sviluppo. **La sezione 6.7** sostiene un approccio evolutivo per stabilire un programma di qualità. Tale approccio ci consente di sviluppare gradualmente competenze nell'uso di dati quantitativi per trovare opportunità di miglioramento dei processi.

Infine, abbiamo abbozzato il framework di maturità del software sviluppato dal Software Engineering Institute. Questo quadro offre un mezzo per valutare lo stato della pratica dell'ingegneria del software, nonché una serie di passaggi per migliorare il processo di sviluppo del software. Uno dei maggiori contributi di CMM e iniziative simili è la loro focalizzazione sul *miglioramento continuo.* Questa linea di pensiero è stata successivamente applicata con successo ad altre aree, risultando, tra le altre, una People-CMM, una Formal Specifications-CMM e una Measurement-CMM.

ULTERIORI LETTURE

Fenton e Pfleeger (**1996**) forniscono una panoramica molto completa del campo delle metriche software. (Endres e Rombach, **2003**) è un libro di testo di ingegneria del software incentrato su

osservazioni e misurazioni empiriche. Il quadro di misurazione discusso nella **Sezione 6.1** si basa su (Kitchenham *et al.*, **1995**). Kaner e Bond (**2004**) forniscono anche una struttura per la valutazione delle metriche. (Kitchenham *et al.*, **2007**) discutono degli errori comuni commessi quando si applicano le metriche del software. (Software, **1997b**) e (JSS, **1995a**) sono numeri di riviste speciali sulle metriche del software. Molti degli articoli in questi numeri trattano dell'applicazione delle metriche nei programmi di qualità.

Una delle prime pubblicazioni importanti sul tema dei programmi di misurazione è (Grady e Caswell, **1987**). I fattori di successo per i programmi di misurazione possono essere trovati in (Hall e Fenton, **1997**) e (Gopal *et al.*, **2002**). Pfleeger (**1995**) approfondisce la relazione tra programmi di metrica e livelli di maturità. Niessink e van Vliet (**1998b**) forniscono un framework simile a CMM per la capacità di misurazione delle organizzazioni di software.

Le tassonomie più note degli attributi di qualità del software sono riportate in (McCall *et al.*, **1977**) e (Boehm *et al.*, **1978**). Gli attributi di qualità ISO sono descritti in (ISO 9126, **2001**) e (Côté *et al.*, **2006**). Discussioni critiche di questi schemi sono fornite in (Kitchenham e Pfleeger, **1996**) e (Fenton e Pfleeger, **1996**). (Suryn *et al.*, **2004**) forniscono una panoramica di ISO/IEC 90003.

Garvin (**1984**) fornisce definizioni di qualità. Diversi tipi di benefici in una definizione di qualità basata sul valore sono discussi in (Simmons, **1996**). Per un'elaborata discussione sul Total Quality Management, vedere (Bounds *et al.*, **1994**) e (Ishikawa, **1985**).

Il Capability Maturity Model (CMM) si basa sul lavoro seminale di (Humphrey, **1988**) e (Humphrey, **1989**). Per una descrizione completa del Capability Maturity Model Integrated (CMMI), vedere (CMMI Product Team, **2006**). Le esperienze pratiche con i programmi di miglioramento dei processi software sono discusse in (Wohlwend e Rosenbaum, **1994**), (Diaz e Sligo, **1997**), (Fitzgerald e O'Kane, **1999**), (Conradi e Fuggetta, **2002**) e (Dybå, **2005**). Rainer e Hall (**2003**) discutono dei fattori di successo e Baddoo e Hall (**2003**) discutono dei demotivatori per SPI. (Basili *et al.*, **2002**) discutono delle lezioni apprese da 25 anni di miglioramento dei processi alla NASA. Un'analisi dei benefici e dei costi dei programmi di miglioramento dei processi software è fornita in (Herbsleb *et al.*, **1997**) e (Gartner, **2001**). Le organizzazioni di livello 5 di CMM ad alta maturità sono discusse in (Software, **2000**) e (Agrawal e Chari, **2007**).

Il Personal Software Process (PSP) è descritto in (Humphrey, **1996**) e (Humphrey, **1997a**). BOOTSTRAP è descritto in (Kuvaja *et al.*, **1994**) e SPICE in (El Emam *et al.*, **1997**).

Le critiche agli approcci simili a CMM si trovano in (Fayad, **1997**) e (Fayad e Laitinen, **1997**). El Emam e Madhavji (**1995**) discutono dell'affidabilità delle valutazioni di processo.

Il miglioramento dei processi è l'argomento di diversi numeri speciali di riviste; si veda (CACM, **1997**) e (Software, **1994a**). La rivista *Software Process: Improvement and Practice* è interamente dedicata a questo argomento.

Esercizi

1. Definire i seguenti termini: misura, misura, metrica.

2. **Qual è la differenza tra un attributo interno e uno esterno?**

3. Definire il termine *condizione di rappresentazione.* Perché è importante che una misura soddisfi la condizione di rappresentazione?

4.Qual è la differenza principale tra una scala ordinale e una scala a intervalli? E tra una scala di intervalli e una scala di rapporti?

5.**Quali sono le principali differenze tra le definizioni di qualità basate sull'utente e basate sul prodotto?**

6.Quali sono le tre categorie di fattori di qualità del software distinti da McCall?

7.Discutere la visione trascendente della qualità del software.

8.Quale delle definizioni di qualità di Garvin è utilizzata principalmente dallo sviluppatore di software? E quale è maggiormente utilizzato dall'utente?

9.Quale punto di vista della qualità è sottolineato dalla ISO 9126?

10.Discutere i capisaldi del Total Quality Management.

11.Qual è lo scopo della garanzia della qualità del software?

12.Perché l'organizzazione di garanzia della qualità del software dovrebbe essere indipendente dall'organizzazione di sviluppo?

13.Perché i membri del progetto dovrebbero ricevere feedback sull'uso dei dati sulla qualità che inviano al gruppo di garanzia della qualità?

14.Descrivere i livelli di maturità del Capability Maturity Model.

15.Qual è la principale differenza tra il livello 2 e il livello 3 del Capability Maturity Model?

16.Qual è la differenza tra le versioni a fasi e continue di CMMI?

17.Perché è importante quantificare i requisiti di qualità?

18.♠Considera un progetto di sviluppo software in cui sei stato coinvolto. Come è stata gestita la qualità in questo progetto? I requisiti di qualità sono stati definiti in una fase iniziale? Questi requisiti sono stati definiti in modo tale da poter essere testati in una fase successiva?

19.♡Definire le proprietà misurabili di un prodotto software che costituiscono i criteri di qualità Modularità e Operabilità. Queste proprietà costituiscono una misura oggettiva di questi criteri? In caso negativo, in che modo viene introdotta la soggettività?

20.♡Dare un possibile personale per un gruppo SQA, sia per una piccola organizzazione di sviluppo (meno di 25 persone) che per una grande organizzazione di sviluppo (più di 100 persone).

21. ♠ Elabora un piano di garanzia della qualità per un progetto in cui sei stato coinvolto.

22. ♠ Un requisito di qualità spesso affermato è che il sistema dovrebbe essere "facile da usare". Discutere le possibili differenze tra il punto di vista dello sviluppatore e il punto di vista dell'utente nella definizione di questa nozione. Pensa a modi alternativi per definire l'usabilità del sistema in termini misurabili.

23. ♠ Utilizzando la classificazione del Capability Maturity Model, determinare il livello di maturità che meglio si adatta alla propria organizzazione. Quali passaggi proporresti per far avanzare l'organizzazione a un livello di maturità superiore? Si stanno portando avanti delle azioni per passare dal livello attuale a uno più maturo?

24. ♠ Scrivi un saggio critico sulla valutazione della maturità del software, come esemplificato dal Capability Maturity Model. L'ulteriore sezione di lettura fornisce ampie indicazioni alla letteratura su questo argomento.

25. ♡ Discutere le differenze tra gli approcci SPI per le grandi e piccole imprese (vedi anche (Conradi e Fuggetta, 2002)).

26. ♡ Nel 1988 e nel 1998 sono state condotte indagini per valutare lo stato dell'arte nella stima dei costi del software nei Paesi Bassi. Una delle domande riguardava i tipi di stakeholder coinvolti nello sviluppo di una stima dei costi. Le percentuali risultanti sono state le seguenti:

	1988	1998
Gestione	48.9	75.8
Dipartimento del personale	22.8	37.4
Team di sviluppo	22.6	23.6
Responsabile del progetto	36.7	42.3
Cliente	15.4	15.9
Altro	8.9	8.2
Numero medio di parti coinvolte	1.55	2.03

Si è concluso che lo stato dell'arte per quanto riguarda la stima dei costi è migliorato. Nel 1998 il numero medio delle parti coinvolte è aumentato e questo è stato ritenuto un buon segno. Per ogni singola categoria è cresciuta anche la percentuale di stakeholder coinvolti.

Riuscite a pensare a una conclusione possibilmente negativa da questo stesso insieme di dati, ovvero che la situazione è *peggiorata* dal 1988?

[1] Il Capability Maturity Model e il CMM sono registrati presso l'US Patent and Trademark Office.

[2] CMMI è un marchio di servizio della Carnegie Mellon University.